



MIGRATING ZEC BETWEEN SHIELDED POOLS

SPROUT → SAPLING → IRONWOOD

SAPLING → IRONWOOD

A practical guide for Sprout and Sapling-era holders: locating legacy keys, crossing the turnstile, and reaching Ironwood.

DON'T PANIC!

Bring backups, disk space, and finish Sprout recovery before the NU7 V4 cutoff.

UPDATED: SEPT 18th 2026 • SPROUT CUTOFF TARGET: NOV 5th 2026

Welcome back, Zcasher!

If your ZEC has been sitting untouched since the Sprout days, a **lot has changed** - new pools, new node software, a whole new stack - **but the mission hasn't**. This guide exists to help you find your way back to it, one careful step at a time.

If you never left but have been putting the migration off, this guide is also for you!

Meet Jubjub. The name is not just a mascot. It also belongs to the elliptic curve that has secured every Sapling shielded transaction since 2018. The Zcash Protocol Specification named that curve after the bird in Lewis Carroll's Jabberwocky.



Purpose and limits

This guide is written first for people who may still control Sprout ZEC created in the 2016-2018 era. It explains recovery processes in some detail, but is not a guarantee that any wallet, sidecar, snapshot, or future network proposal will remain available.

Back up wallet.dat, exports, and standalone keys before doing anything. Never paste private keys into a website or chat. Oh, and did I already mention backing up wallet.dat, exports, and standalone keys?

Contents



Part I The brief user guide - DON'T PANIC!

- 1 The pools, without the cryptographic luggage
- 2 The dates that matter
- 3 Migration routes, tools, and costs
- 4 Disk, time, and hardware
- 5 After the Sprout migration: cleanup and safe shutdown

Part II Technical migration reference

- 6 Pool and transaction architecture
- 7 Sprout → Sapling: what must exist and run
- 8 Sapling → Ironwood: modern wallet migration
- 9 Alternative routes and privacy trade-offs
- 10 Where Zebra, Zakura, the sidecar, and modern wallets fit
- 11 ZIP 2003: NU7 Sprout freeze and future retrieval
- 12 Post-quantum recoverability and long-term storage
- 13 Operational checklist and troubleshooting

Appendix

- 14 Copied datadirs and recovery snapshots

Also in the Appendix:

command and configuration skeleton,

Stuck? Need help?

glossary, scenario matrix, source notes,

disclaimer, and change history.

PART I

DON'T PANIC!

A brief guide for people who want their ZEC moved safely, not a dissertation on zero-knowledge circuits.

START HERE

The answer in one page



The blockchain is large, the terminology is new, and the family tree has more branches than is polite. Nevertheless, the practical choices are clear.

If your funds are in Sprout

Use the dedicated Sprout-to-Sapling migration functionality in the constrained zcashd sidecar behind Zebra or Zakura while V4 format transactions remain valid. Do not begin with a manual `z_sendmany` sweep. After the Sapling notes confirm, use a modern Ironwood-capable wallet for the second stage. Bring backups, disk space, proving parameters, and tea. [4][8][9][16]

Before building that rig, **check Section 3: the Argos rescue wallet now reads wallet.dat directly** and may recover Sprout at a fraction of the disk and time cost. [74][75]

If your funds are already in Sapling

You do not need Sprout code or wallet.dat archaeology. A modern Ironwood-capable wallet can create one V6 format transaction that spends Sapling notes and creates Ironwood notes. This guide includes that second stage because it is the natural destination after Sprout recovery. [10][12]

Sprout cutoff: NU7 target Nov 5th 2026

NU7 is targeted for Mainnet activation on Nov 5th 2026. Its announced scope **disables V4 transactions**, which freezes today's Sprout spending path. The exact activation height will be set on Oct 20th after Testnet activation; about block 3,543,000 is a planning estimate only. If you are **reading after the actual activation height**, existing tools cannot move Sprout into Sapling, transparent funds, or any other destination. At that time the Sprout pool is frozen in ice. [11][79][80]

A five-question decision tree

- 1. Is it Sprout?** A seed phrase alone means Sapling or Orchard, not Sprout: go to Section 3, "Direct if not Sprout," and skip Part II. A `zc` address, or a restored wallet reporting a Sprout balance, means Sprout.
- 2. What can you recover it with?** `wallet.dat`, the old computer or `datadir`, a `z_exportwallet` backup, or an exported Sprout spending key. A `zc` address alone is not enough.
- 3. Argos or the legacy sidecar?** Both are Sprout-capable. Section 3 compares them.
- 4. Do you already have a synchronized, unpruned zcashd datadir?** Copy it only after a clean shutdown. Otherwise download a snapshot, which saves days or weeks.
- 5. Where do the funds end?** Always Ironwood. Sprout crosses through Sapling first, because no single transaction can go direct. Do not stop at Sapling.

Before you commit: is the ZEC really there, and yours to spend?

Two disappointments account for a lot of wasted effort: the wallet **holds no spendable Sprout value**, or it holds viewing material **rather than a spending key**. Both are visible before you migrate, and the first check needs no synchronized chain at all.

If you hold wallet.dat or a standalone Sprout spending key, read the Argos option in Section 3 first: Argos answers both questions as part of its normal flow.

If not using Argos, install the sidecar and load the wallet, but do not start the sync yet. Run the Stage 1 commands first. Syncing is what costs 550 GiB to 1 TiB and days to weeks (Sections 4 and 14), so it is worth knowing what the wallet holds before you begin.

These commands assume the sidecar is running and zcash-cli can reach it. See the command and configuration skeleton in the Appendix if it returns a connection or authentication error.

Stage 1, as soon as the wallet loads. The sidecar does not have to be synced to answer whether this wallet contains Sprout addresses and whether you hold the keys that spend them. These commands assume the sidecar is running and zcash-cli can reach it. See the command and configuration skeleton in the Appendix if you get a connection or authentication error.

```
# Are there any Sprout addresses here? zc... = Sprout, zs... = Sapling
zcash-cli z_listaddresses

# Do you hold the SPENDING key, or only a viewing key?
# This fails if the wallet has viewing material only.
zcash-cli z_exportkey "zc..."
```

Stage 2, after the scan finishes. Balances only mean something once the sidecar has scanned from before your first Sprout receipt. Expect hours at best, and days if you are scanning the full Sprout range with many old addresses on slow storage. The cost scales with how far back your earliest receipt sits and how many addresses must be trial-decrypted, not with how much ZEC you hold.

```
# unmigrated_amount is the Sprout value the scheduler still has to move
zcash-cli z_getmigrationstatus

# Per-note detail, including which address each note belongs to
zcash-cli z_listunspent
```

- **A Sprout balance is not proof of spending authority.** `z_gettotalbalance` reports Sprout and Sapling together as one "private" figure. Its own documentation warns that this figure can exceed what you can actually spend. When the wallet holds only incoming viewing keys, it cannot see spends made by whoever holds the real key. Trust `z_exportkey` and per-address checks over the headline total. [17][19][67]
- **unmigrated_amount is the number that matters.** It is the Sprout value the ZIP 308 scheduler would still move. A `zc` address with zero here usually means the notes were already spent, or that the wallet holds viewing material only. [4][18]
- **Some of these RPCs are deprecated.** Recent `zcashd` releases gate `z_getbalance`, `z_gettotalbalance`, and `z_listaddresses` behind `-allowdeprecated`. If a command returns a deprecation error, enable that specific feature rather than concluding the wallet is empty. [67]
- **A zero result is not always final.** An incomplete rescan looks exactly like an empty wallet. Confirm the sidecar has finished scanning the range that contains your original Sprout receipts before believing a zero. [16]

Cheap to check, expensive to assume

Stage 1 costs minutes. A week of synchronization can end in one of two discoveries: the wallet carries only a viewing key, or the Sprout notes were spent years ago. Both are avoidable. Checking first costs nothing, and saves the time and the disk you would otherwise have bought.

A note about who used Sprout

Sprout holders were often **early adopters** and technically capable, but some used **consumer-style desktop packages**. These included zcashd/zcash-cli, Zcash Swing or Desktop GUI, Zcash4Win, Zcash4Mac, WinZEC, and ZecWallet Fullnode. Most were interfaces wrapped around the same zcashd internal wallet. So the decisive recovery object is usually wallet.dat, or an exported Sprout spending key, not an old GUI installer. [13][14][19]

Before Sapling, mobile Zcash wallets existed, but they **supported only transparent ZEC**; practical shielded mobile wallets arrived with Sapling-era light-wallet technology. Pre-Sapling, mobile wallets such as Coinomi and Jaxx supported only transparent ZEC. Their users therefore do not need Sprout migration tooling; recovery is a normal seed/private-key problem. Jaxx Liberty has been retired, while Coinomi remains supported. [63]

Wallet remembered by the user	What it usually used underneath	Recovery material to look for
zcashd / zcash-cli	The native zcashd full node and internal wallet	wallet.dat, the complete datadir, z_exportwallet output, or z_exportkey output
Zcash Swing / Desktop GUI; Zcash4Win; Zcash4Mac; WinZEC	A graphical wrapper or packaged build around zcashd	The same wallet.dat and Zcash datadir used by the bundled node
ZecWallet Fullnode / zec-qt-wallet	A later full-node interface that could reuse earlier zcashd data	Existing wallet.dat/datadir, or exported legacy keys
Lite/mobile wallet with only a mnemonic	Usually a later Sapling/Orchard wallet. Could also be transparent ZEC only	Probably not a Sprout shielded wallet; use the wallet's own modern recovery path

A recent successful recovery before the release of Argos began from imported private material rather than the original wallet file. The user had to distinguish the Sprout spending key from the public zc address, complete a historical scan, install the Sprout proving parameters, and then use the dedicated migration commands. [16]

A zc address proves only that the destination is Sprout. It does not prove that you possess the spending key. Preserve the old storage device before trying repair tools, and work only on copies.

Historical software is evidence, not a recommendation

Old GUI wallet installers may contain obsolete dependencies or unsafe update paths. Preserve the wallet.dat, but use a current, reviewed recovery environment.

Why Sprout has to go

The Sprout shielded pool is obsolete, but Zcash validators must still support its JoinSplit machinery, commitments, nullifiers, anchors, value-pool rules, and historical formats - adding complexity, testing burden, and attack surface.

Sprout is also not quantum-recoverable: Groth16 soundness and its elliptic-curve signatures would become insecure against a sufficiently capable quantum attack.

Since Sapling activated, both pools have used Groth16 over BLS12-381, so retiring Sprout does not remove that proof system. It does remove the substantial consensus code unique to Sprout.

ZIP 2003 is now in the announced NU7 scope. Its goal is to remove Sprout-specific complexity and attack surface by disallowing V4 transactions. That also disables Sprout spending. NU7 Mainnet is targeted for Nov 5th 2026; the exact activation height is due Oct 20th. Sapling will need its own retirement path later. [11][79][80]

Why Sapling will also have to go

Sapling's shielded pool still works, but it was not designed for indefinite use or long-term cold storage. Its note commitments depend on elliptic-curve discrete-logarithm assumptions and are not quantum-recoverable.

If those assumptions become breakable, Sapling cannot simply be upgraded in place: spending from the pool may need to be disabled to prevent theft or forged funds. Sapling will probably never be quantum-recoverable, and ZIP 2005 says funds must move into Ironwood to gain that protection.

Retiring Sapling would eventually remove its remaining legacy proof and transaction machinery from live consensus. Unlike Sprout, Sapling currently has no announced shutdown date.

Why Orchard had to go

On July 28th 2026, **Orchard effectively become exit-only** with the NU6.3 upgrade. No new value could enter the pool, ordinary cross-address Orchard payments were disabled. Existing Orchard funds need to be migrated into the new Ironwood pool. Wallets need Ironwood pool support to perform that migration. [23]

This followed Taylor Hornby's discovery on **May 29th 2026** of a critical Orchard circuit bug that could permit valid-looking proofs for invalid state transitions. He demonstrated counterfeit ZEC in a test environment; the flaw had existed since Orchard launched in 2022. [44][45][47]

NU6.2 patched the bug, with no evidence of Mainnet exploitation. [44][46] But past shielded activity cannot be retrospectively proven sound. The Ironwood pool therefore provides a clean break: Orchard's turnstile limits how much value can leave, bounding any possible hidden inflation before funds enter the new pool. [45][23]

1. The pools, without the cryptographic luggage



Pool	Plain-English picture	What matters for migration
Transparent	A glass box: addresses, inputs, outputs, and amounts are public.	Useful as an emergency bridge, but creates a linkable, publicly viewable transaction history. Like Bitcoin.
Sprout	The original Zcash shielded pool, introduced at launch in 2016.	Legacy JoinSplit/V4 format only. Exit-only since the Canopy network upgrade. Requires legacy wallet/prover code.
Sapling	A lighter, more practical shielded pool introduced in 2018.	The official Sprout migration destination; coins in Sapling can move directly to Ironwood in V6 format.
Orchard	A newer shielded pool built from uniform "Actions."	Closed to new deposits and internal transfers after a 2026 soundness bug; exit only, via Ironwood. See "Why Orchard had to go" above.
Ironwood	A new shielded pool using the Orchard protocol with recoverable note construction.	The destination for shielded value after NU6.3.

A "pool" is a separately accounted set of value with its own rules/cryptographic state. Moving between shielded pools publicly changes both pool balances, revealing the amount transferred but not who sent or received it.

The transparent pool reveals the source, destination, and amount, much like Bitcoin.

2. The dates that matter



Date	Block height	Milestone	Meaning for Sprout and migration
Oct 15th 2016	-	Early evolution warning	Before launch, Zcash designers publicly warned that future upgrades might require action to move dormant funds. This established that coins in old pools would not remain spendable forever. [20]
Oct 28th 2016	0	Zcash launches with Sprout	Sprout is the original shielded protocol and zc addresses are its payment addresses.
Oct 28th 2018	419,200	Sapling activates	Faster, lower-memory shielded payments become available; Sprout remains spendable. [1]
Nov 27th 2018	-	ZIP 308 created	The privacy-preserving Sprout-to-Sapling migration procedure and supporting wallet RPCs are specified. [4]
Mar 29th 2019	-	ZIP 211 created	The formal first step toward removing Sprout is proposed: stop adding new value to its chain value pool balance. [2]
May 20th 2019	-	zcashd 2.0.5-2	Official Sprout-to-Sapling migration tool released. [5]
Nov 18th 2020	1,046,400	Canopy + ZIP 211	The ability to add new value to the Sprout pool balance is disabled. Sprout becomes exit-only. Existing coins may leave. [2][3]
May 31st 2022	1,687,104	Orchard NU5 + V5 format	Orchard and its V5 format omits Sprout, but the V4 format remains valid for Sprout withdrawals and migration. [1]
Mar 31st 2026	-	zcashd 6.12.0	Sprout-to-Sapling migration rounds are accelerated from every 500 blocks to every 100 blocks, and the average amount above 1 ZEC rises from about 5 ZEC to about 500 ZEC. This shortens the time needed to migrate. [6][25]
July 18th 2026	3,417,100	Stock zcashd halt	Stock zcashd binaries stop and refuse restart. This is a software halt, not a V4 format consensus cutoff. [7]
July 28th 2026	3,428,143	NU6.3 + V6 format + Ironwood activated	Ironwood is live. V4, V5, and V6 transactions coexist; Sprout remains spendable through V4 until NU7 activates. The announced Mainnet target is Nov 5th 2026. [10][23][28][79]
Nov 5th 2026 (target)	~3,543,000*	NU7: V4 disabled / Sprout frozen	Target announced Sept 17th. NU7 includes disabling V4 transactions, so Sprout spending stops at activation. Testnet target: Oct 6th. Final Mainnet decision and exact activation height: Oct 20th. *~3,543,000 is a planning estimate only, not the consensus height. [11][79][80]

Sprout deprecation happened in stages. ZIP 211, deployed with Canopy, made the pool aggregate exit-only and explicitly described this as a step toward removing Sprout. ZIP 2003 is now in the announced NU7 scope: at NU7 activation, V4 transactions are disabled and Sprout spending freezes. The Mainnet target is Nov 5th 2026, with the exact activation height due Oct 20th. [2][3][11][79][80]

How much time do I actually have for Sprout migration?

Two clocks run at different speeds, and the slower one is usually yours.

The network clock is now counting down. On Sept 17th, Zcash ecosystem engineering teams announced an NU7 Mainnet target of Nov 5th 2026, with V4 transactions disabled. Testnet targets Oct 6th; the final Mainnet decision and exact activation height are due Oct 20th. As of this Sept 18th edition, that target is 48 days away. The ~3,543,000 height used in this guide is only a planning estimate until the official height is set. [11][79][80]

Your own clock is the binding constraint. Building or restoring legacy state can take days to weeks, although Argos may be much faster for compatible recovery material. A from-scratch sidecar sync can run to several weeks on modest hardware. One documented recovery needed repeated chain rebuilds, then six days for 21 scheduled migration transactions. ZIP 308 is paced by block height, not by your deadline. Start now and aim to finish and confirm well before NU7 activation. [6][9][16][79]

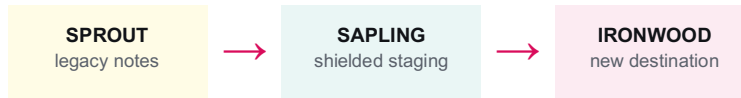
SPROUT FREEZES IN NOVEMBER. START NOW.

Mainnet NU7 target: Nov 5th 2026. Planning height: ~3,543,000; the exact consensus activation height is due Oct 20th. At activation, V4 transactions stop and today's Sprout exit paths stop with them. Aim to finish and confirm days ahead, not at the cutoff. [11][79][80]

3. Migration routes, tools, and costs



Preferred: Sprout → Sapling → Ironwood



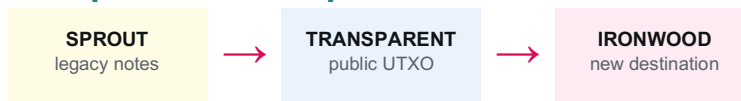
Two transactions and two visible pool-balance crossings; the middle note remains shielded.

- Stage 1 uses the constrained zcashd wallet to create V4 format Sprout-to-Sapling migration transactions.
- Stage 2 uses a modern Ironwood wallet to create a V6 format Sapling-to-Ironwood transaction.
- Do not immediately transfer the same unique amount from Sapling to Ironwood if privacy matters; delay, split, or combine according to the modern wallet's migration policy.

100 ZEC example: 100.0000 Sprout → 99.9999 Sapling → 99.9998 Ironwood.

Best-case end-to-end: 1 day. **Minimum:** 0.01 ZEC.

Simpler but public: Sprout → transparent → Ironwood



The transparent outpoint explicitly links the two legs. Random timing cannot erase that link.

- This is technically possible while V4 format is valid, because the new zcashd sidecar can construct the first leg.
- It is an emergency compatibility route, not the privacy-preserving default.

100 ZEC example: 100.0000 Sprout → 99.99985 transparent → 99.99975 Ironwood.

Best-case end-to-end: 1 day. **Minimum:** more than 0.00025 ZEC.

Which route should a wallet.dat holder take?

If you hold wallet.dat or a standalone Sprout spending key, and you just want the funds out, **try Argos first**. The difference in disk, bandwidth, and time is close to two orders of magnitude. The Part II sidecar path remains the reference route. Fall back on it if Argos cannot read your material, if you would rather not put spending keys into third-party software, or if you want every component under your own control. Either way, the destination checks and post-migration hygiene in Section 5 apply unchanged.

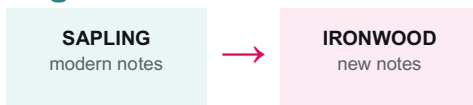
A lighter path for wallet.dat holders: the Argos rescue wallet

Since v1.2.0 (Sept 1st 2026), the Argos rescue wallet from Sovright has been able to read legacy zcashd wallet.dat files directly and recover transparent, Sapling, and Sprout funds. Argos also supports Orchard recovery through its seed-based recovery path. For a holder who simply wants old funds out, it can stand in for the full-node sidecar rig described in Part II, with a fraction of the storage footprint and setup overhead. It is the most significant change to the recovery landscape since this guide was first written. [74][75]

- **Which Argos version? Current as of Sept 18th is v1.3.0 (released Sept 16th). Use v1.2.0 or later for wallet.dat and Sprout recovery; v1.3.0 is preferred because it expands recovery coverage, improves Sapling matching, updates security dependencies, and fixes a sweep that could report success without moving funds to the destination. Older builds that construct pre-Ironwood transactions must be updated before retrying.** [75]
- **The cost difference is the headline.** If a wallet.dat contains spendable Sprout note data and a cached witness, Argos can sweep directly without a historical scan. If only the Sprout spending keys remain, it must locate the notes by trial-decrypting historical JoinSplits, a block scan from genesis through Canopy that is roughly 26 GB of network transfer. Argos says the process takes hours; **six to 24 hours is a reasonable planning allowance, depending on host and peer/network performance.** Compare that with the 550 GiB to 1 TiB footprint and multi-week synchronization described in Section 4. The Argos scan is checkpointed and resumable, with less than 500 MB retained on disk. [75]
- **It reads encrypted wallet files, and standalone keys.** Argos opens encrypted zcashd wallet.dat files directly, and also accepts standalone Sapling or Sprout spending keys of the SK or ST form. Note that a ZecWallet Lite seed alone cannot reconstruct Sprout keys, because that wallet generated them independently. [75]
- **It is credentialed, but it is still third-party software holding your keys.** Argos is a local-first desktop application with a Rust backend, published as open source, and independently audited by Least Authority with every finding in the final report marked resolved. Seeds and spending keys stay on your machine: only public block data comes in and signed sweeps go out. That is a stronger profile than most recovery tooling. It is not a reason to skip the machine hygiene in Section 13, or to install from anywhere other than the project release page with checksums verified. [75]
- **The sweep step offers a donation, defaulting to 10 percent.** Recovery itself is free. The sweep review screen includes an optional donation to Sovright, pre-set to 10 percent, with both outputs displayed before you confirm. Set it to whatever you think is right, including zero, but do not click through that screen without reading it. [75]
- **Internet access is required.** Air-gapped recovery is not possible here: ordinary scans use lightwalletd and Sprout scans pull historical blocks from public P2P peers. [75]

Argos automates a path; it does not repeal consensus. A Sprout exit still leaves through V4, the same consensus path used by the sidecar. NU7 is targeted for Nov 5th 2026 and is announced to disable V4, so Argos cannot rescue Sprout by transaction construction after the actual NU7 activation height. What Argos changes is the recovery machinery you have to operate before that cutoff. [11][75][79]

Direct if not Sprout: Sapling → Ironwood



One V6 format transaction can contain Sapling spends and Ironwood Actions.

- No zcashd datadir, Sprout parameters, or JoinSplit prover is needed for Sapling to Ironwood.
- The wallet must support both the source Sapling account and Ironwood transaction construction.

100 ZEC example: 100.0000 Sapling → 99.9999 Ironwood.

Best-case end-to-end: 0.5 day (12 hours). **Minimum:** more than 0.0001 ZEC.

Direct if not Sprout: Orchard → Ironwood



Orchard is exit-only after NU6.3. A current wallet can move Orchard directly to Ironwood.

- No zcashd datadir, Sprout parameters, or JoinSplit prover is needed for Orchard to Ironwood.
- Choose a wallet with ZIP 318 support for privacy-preserving scheduled, or immediate, coin migration.
- The wallet must support both the source Orchard account and Ironwood transaction construction.

100 ZEC example: 100.0000 Orchard → 99.9999 Ironwood.

Best-case end-to-end: 0.5 day (12 hours). **Minimum:** more than 0.0001 ZEC.

Fees and small-balance cutoffs

These are planning figures, not guaranteed wallet quotes. ZIP 317's conventional fee scales with logical actions, so 0.0001 ZEC is the **conventional minimum for an ordinary transaction**, not a universal fixed fee. The 0.01 ZEC figures in ZIP 308 and ZIP 318 are migration/privacy cutoffs, not 0.01 ZEC transaction fees. [4][24][51]

If you recover with Argos, read the sweep screen. Argos adds a cost that is not a network fee. Its sweep screen offers an optional donation to Sovright, pre-set to 10 percent of the recovered amount. That dwarfs every figure in this section: on a 100 ZEC recovery the default donation is 10 ZEC, against roughly 0.0002 ZEC in actual transaction fees. Both outputs are shown before you confirm, and you can set the donation to any value including zero. Read that screen. [75]

Route / leg	Fee planning	Small-balance rule
Sprout → Sapling (ZIP 308 scheduler)	0.0001 ZEC per migration transaction, fixed by ZIP 308. The scheduler may create more than one transaction. [4]	The scheduler stops when less than 0.01 ZEC remains. If the whole Sprout balance is below 0.01 ZEC, this scheduler route will not create a migration transaction.
Sprout → Sapling → Ironwood	Stage 1: $N \times 0.0001$ ZEC. Stage 2: a normal ZIP 317 wallet fee; the conventional floor is 0.0001 ZEC, but extra logical actions or padding can make it higher. [4][51]	The starting Sprout balance must reach the 0.01 ZEC ZIP 308 threshold. The resulting Sapling value must also exceed the second-stage wallet fee.
Sprout → transparent → Ironwood	Manual route. Under ZIP 317, the simplest one-JoinSplit Sprout→transparent leg is about 0.00015 ZEC (2 Sprout logical actions + 1 transparent output). A minimal transparent→Ironwood leg has a 0.0001 ZEC conventional floor. Illustrative two-leg floor: about 0.00025 ZEC; actual fees can be higher. [51]	This bypasses ZIP 308's 0.01 scheduler cutoff, but it sacrifices privacy. The balance must exceed the combined actual fees; at or below the fee total, no positive value can arrive in Ironwood.
Sapling → Ironwood	Normal ZIP 317 transaction. The conventional minimum is 0.0001 ZEC; the actual fee can be higher because Sapling and Ironwood logical actions are both counted. Use the wallet's quoted fee. [51]	No special 0.01 cutoff. The source balance must exceed the actual constructed fee; near that level, the value is effectively dust.

Route / leg	Fee planning	Small-balance rule
Orchard → Ironwood (ZIP 318 private schedule)	Each canonical pool-crossing part uses the canonical fee, provisionally the ZIP 317 minimum (0.0001 ZEC). Multiple parts multiply that cost; note-preparation transactions add their own ZIP 317 fees when needed. [24][51]	The smallest canonical denomination is 0.01 ZEC and its fee is additional. About 0.0101 ZEC is therefore the theoretical one-part floor if no note preparation is needed; preparation can raise the practical minimum. A residual below 0.01 ZEC is left in Orchard.
Orchard → Ironwood (immediate)	One ordinary cross-pool transaction priced by ZIP 317. Conventional floor: 0.0001 ZEC; action count/padding can raise it. [24][51]	No ZIP 318 0.01 private-schedule floor. The Orchard balance must exceed the actual transaction fee.
Transparent → Ironwood	Normal ZIP 317 transaction. Conventional floor: 0.0001 ZEC; more/larger transparent inputs or additional Ironwood actions can raise it. [51]	No 0.01 cutoff. The source value must exceed the actual fee.

Worked fee examples: 100 ZEC and 500 ZEC

These use the simple planning floors above, not a guaranteed wallet quote. The amount of ZEC moved does not by itself raise the ZIP 317 fee: transaction shape, input/action count, and the number of Sprout migration transactions do. For the Sprout route, the examples assume one ZIP 308 migration transaction unless noted. [4][6][51]

These examples assume you are not using Argos. The figures below are network fees only. If you recover through Argos and leave its donation at the default 10 percent, about 90 ZEC of a 100 ZEC recovery and about 450 ZEC of a 500 ZEC recovery reach your destination. The donation, not the fee, is then the dominant cost. Set it carefully. [75]

Route	Starting with 100 ZEC	Starting with 500 ZEC
Sprout → Sapling → Ironwood	Simple floor example: 1 ZIP 308 migration tx (0.0001 ZEC) + 1 minimal Sapling → Ironwood tx (0.0001 ZEC) = about 0.0002 ZEC total. About 99.9998 ZEC reaches Ironwood.	Same simple floor: about 0.0002 ZEC total, leaving about 499.9998 ZEC in Ironwood. If the Sprout scheduler needs 2 migration txs, the floor becomes about 0.0003 ZEC; each additional Sprout migration tx adds 0.0001 ZEC.
Sprout → transparent → Ironwood	Simplest two-leg planning floor: about 0.00025 ZEC total (about 0.00015 + 0.0001). About 99.99975 ZEC reaches Ironwood.	Same simplest floor: about 0.00025 ZEC total. About 499.99975 ZEC reaches Ironwood. More JoinSplits, transparent inputs, or Ironwood actions raise the fee.
Sapling → Ironwood	One ordinary V6 transaction: about 0.0001 ZEC conventional floor. About 99.9999 ZEC reaches Ironwood.	Same conventional floor: about 0.0001 ZEC. About 499.9999 ZEC reaches Ironwood. Extra Sapling spends, Ironwood actions, or wallet padding can raise it.
Orchard → Ironwood (ZIP 318 private schedule)	Baseline canonical example: 99.99 ZEC moves in 12 scheduled parts. Migration-part fees total 0.0012 ZEC. Assuming one standard 16-action note-preparation transaction is needed, add about 0.0008 ZEC: about 0.0020 ZEC total fees, 99.99 ZEC reaches Ironwood, and about 0.0080 ZEC remains in Orchard.	Baseline canonical example: 499.99 ZEC moves in 14 scheduled parts. Migration-part fees total 0.0014 ZEC. Assuming one standard 16-action note-preparation transaction is needed, add about 0.0008 ZEC: about 0.0022 ZEC total fees, 499.99 ZEC reaches Ironwood, and about 0.0078 ZEC remains in Orchard.
Orchard → Ironwood (immediate)	One ordinary Orchard → Ironwood V6 transaction at the 0.0001 ZEC conventional floor. About 99.9999 ZEC reaches Ironwood.	Same conventional floor: about 0.0001 ZEC. About 499.9999 ZEC reaches Ironwood. This is the fast one-shot path; the ZIP 318 private schedule uses multiple parts and therefore multiple fees.

Why 100 ZEC and 500 ZEC can have the same floor: Zcash conventional fees are based on transaction structure rather than a percentage of the value transferred. A larger balance costs more only when it requires additional migration transactions, inputs, or logical actions.

What is “too small to bother”? There is no single economic cutoff for ordinary transactions. The hard and procedural limits are clearer. ZIP 308’s Sprout scheduler does nothing below 0.01 ZEC. ZIP 318’s private Orchard flow uses 0.01 ZEC as its smallest denomination and adds the fee on top. Every manual route needs enough source value to pay its actual transaction fees. If fees would consume most or all of a balance, that balance is effectively dust. [4][24][51]

Network-level exposure: shielding coins does not hide your IP

Shielding hides what is in your transaction, not where it came from. The server you connect to, and anyone watching the link, can tie your IP address to the fact that you are transacting and to the timing of each step. A migration running in batches over hours or days leaves a long observation window, and the cadence of those batches is itself a pattern. [76]

- **Use a network privacy tool for the whole run, not just the first transaction.** The exposure lasts as long as the schedule does. Wallet developers raised this point during the Ironwood migration. Nym announced free NymVPN access for the migration in July 2026; check whether that offer still stands before relying on it. Tor, or any other trusted network-privacy layer, serves the same purpose. [76]
- **Running your own front node helps, but does not remove the problem.** The Part II arrangement means you are not handing a third-party light-wallet server your queries, which closes one exposure. Your node still connects to public peers, so the network-layer question remains and should be handled separately.
- **Treat it as part of the threat model, not an afterthought.** A migration performed with perfect on-chain hygiene and a bare residential IP has simply moved the disclosure from the chain to the network. Decide which of the two matters for your situation before you start, because the schedule is hard to change once it is running.
- **Several wallets now ship this rather than leaving it to you.** Zodi has built Tor protection into its migration flow since 3.9.0. Zkool added the Nym mixnet as a transport in 6.27.0. NozyWallet treats the broadcast path as part of migration safety: it prefers a local zebra, or an opt-in Nym mixnet, so the submitting host is not your clearnet IP. Tor and I2P are accepted alternatives. The Zkool maintainer's own advice is to use Tor, through embedded arti or an external SOCKS proxy, to hide your IP from the lightwalletd server. Prefer a wallet that does this for you over bolting it on yourself. [77]
- **Understand precisely what a mixnet or VPN buys you.** It breaks the link between the broadcast and your IP. It does not hide the amount: turnstile crossings are public by design, and no network-layer tool changes that. Amount, timing, and IP are three separate leak channels, and closing one does nothing for the other two. [77]

4. Disk, time, and hardware



Who this section is for. These figures describe the constrained zcashd sidecar used for the Sprout path. If your funds are already in Sapling or Orchard, you need none of it, go back to Section 3, 'Direct if not Sprout...'

If you hold wallet.dat or a standalone Sprout spending key, and you just want the funds out, try Argos first. Go back to read 'A lighter path for wallet.dat holders: the Argos rescue wallet' a few pages earlier.

But if you can't, or don't want to, use Argos, then read on. But note that the process is heavier all round.

Migrate with either 1. download zcashd datadir OR 2. start sync from scratch

The CPU and RAM minimums are the same in both scenarios. The big differences are sync time and network traffic - starting from scratch is much more expensive than just downloading a previously synced datadir copy. The recovery system can be deleted after migration and verification. The RAM required is necessary for Sprout JoinSplit proof generation, which this migration process needs. [11]

Two scenarios

Resource	1. With a Mainnet datadir copy	2. Starting from scratch
Total disk needed	550 GiB minimum 1 TiB recommended	550 GiB minimum 1 TiB recommended
zcashd Mainnet state	Existing full stock-format datadir or authenticated snapshot (size varies; allow hundreds of GiB)	Build and validate the full history through the sidecar
Why the copy helps	Skips most of the sidecar initial sync (SAVES days to weeks)	No shortcut; one-peer initial sync is slow and takes days to weeks
CPU	4 logical CPUs minimum	4 logical CPUs minimum
RAM	16 GiB minimum	16 GiB minimum
Storage	SSD/NVMe	SSD/NVMe
Network	Stable broadband Catch-up only	Stable broadband Hundreds of GiB
Time before migration	Usually hours to a few days	Plan for days to weeks

Why not one exact datadir size? Snapshot archives and extracted live datadirs vary because database compaction, stale blocks, logs, and optional indexes differ. Current zcashd-compat preflight requires 275 GiB free per Mainnet datadir mount, or 550 GiB when Zebra state and the zcashd datadir share one mount; 1 TiB combined is recommended. [9]

Why the storage requirement is the same but the pain is not. The SSD/NVMe requirement above stands for both columns, but the two paths stress the disk very differently, and it is worth knowing which one you are in.

- **Starting from scratch is the punishing case.** Initial block download and index construction are dominated by small random reads and writes, as the node validates history and builds its databases. A spinning disk handles that workload worst. Here the difference between SSD and HDD stops being a matter of hours and becomes a matter of weeks. [9]
- **Restoring a snapshot is mostly sequential.** Downloading and extracting a snapshot archive is largely a bulk streaming write, which slower storage handles far better. The heavy random-I/O phase is what the snapshot skips. That is the same reason the snapshot path saves days to weeks in the table above.
- **The requirement does not relax after restore.** Once the node is running, catch-up validation, wallet rescan, and JoinSplit proof generation return to random access on the same databases. Plan for SSD/NVMe for the whole operation; treat any slower configuration as unverified for a deadline-driven recovery, not as a supported one. [9][16]

Leave txindex off. It serves getrawtransaction and block explorers, not the wallet RPCs this migration uses, and it costs extra disk and sync time. If you are adapting an explorer or general-purpose zcash.conf, check txindex=1 has not carried over. [73]

Planning note:

The CPU and RAM minimums are the same in both scenarios. The **differences are sync time and network traffic.** Starting from scratch is much more expensive than just downloading previously synced datadir copy. The recovery system can be deleted after migration and verification.

The constrained zcashd sidecar build preserves the legacy wallet functions needed to recover Sprout funds after stock zcashd stopped operating. The sidecar does not borrow Zebra's chainstate. Zebra is the network-facing consensus implementation, while the sidecar keeps and validates its own stock-format block and chainstate databases. [9]

- The zcashd sidecar still uses wallet.dat for Sprout and Sapling wallet data.
- A modern wallet such as Zodi, Zkool, Zingo, Vzor, or Cake Wallet uses its own wallet database and seed/key-storage instead.

How long from scratch?

Do not expect a fixed number of days. On a good SSD system, full synchronization and wallet scanning may take several days to a few weeks. Old hardware, an HDD, database corruption, reindexing, or repeated restarts can stretch recovery into months. A recent user needed repeated chain rebuilds and then six days for 21 scheduled migration transactions. Start early. [9][16]

- Using an existing datadir copied with both processes stopped is the least unpredictable path.
- A wallet rescan can add hours or days after chain sync, especially with many old shielded addresses.
- Do not start a deadline-driven Sprout evacuation with only enough disk for today's exact byte count.

Beware of insufficient disk space!

A 320 GB hard-disk setup failed for a recent Sprout recovery when database work, proving parameters, and low free space collided to thwart the full sync. Use 550 GiB as the combined planning floor for Zebra plus sidecar state, and treat 1 TiB as the recommended practical target with safer headroom. [9][16]

5. After the Sprout migration: cleanup and safe shutdown



Do NOT delete wallet.dat as soon as the migration finishes. First confirm exactly where the funds went, that every migration transaction is finalized, and that you can spend the resulting Sapling funds from a modern wallet.

DID I MENTION STOP --- DON'T DELETE?!

...If anything below does not match what is expected, **stop**. Keep the original wallet, sidecar datadir, and backups untouched until the discrepancy is understood.

Check the migration result

zcash-cli z_getmigrationstatus

Look at these fields in the output:

Field	What you want to see	What it tells you
destination_address	The Sapling address you expected	This is where the migration sent the Sprout funds.
unfinalized_migrated_amount	0 ZEC	No migration value is still waiting for finality. zcashd treats a migration transaction as finalized at 10 confirmations.
unmigrated_amount	0 ZEC or less than 0.01 ZEC	The scheduled Sprout migration has finished or reached the ZIP 308 stopping threshold. A sub-0.01 ZEC remainder is expected behavior.
finalized_migrated_amount	Approximately the amount you expected	Should match the expected migrated balance, less fees and any sub-0.01 ZEC remainder.
finalized_migration_transactions	One or more, if funds migrated	Number of migration transactions that reached finality.
migration_txids	A list of the migration transaction IDs	Keep these IDs as your migration record.

Confirm you control the Sapling destination

The destination_address shown above must be an address for which you still have spending authority. If you did not set -migrationdestaddress, the migration normally used the Sapling account-0 destination inside the same legacy wallet. In that case, **wallet.dat** may still be the only thing that can spend the migrated Sapling notes.

A quick way to check the destination is z_validateaddress, which confirms the address parses, reports its address_type (sprout or sapling), and reports whether the wallet knows about it at all.

```
zcash-cli z_validateaddress "zs..."

# "isvalid": true the address parses
# "address_type": "..." sprout or sapling
# "ismine": true|false whether THIS wallet tracks the address
```

- **"ismine": false is a decisive stop.** If the wallet does not track the destination at all, do not continue and do not delete anything. You are looking at the wrong wallet, the wrong datadir, or an address that was never yours.
- **"ismine": true is necessary but not sufficient.** It reports that the wallet tracks the address, not that it can spend from it. zcashd can hold an address through a viewing key alone. The release notes call these addresses watch-only: they see receipts but cannot spend.

zcashd keeps a separate internal check for spending-key ownership, because wallet membership and spending authority are different questions. This is the same trap that lets `z_gettotalbalance` report more than you can spend. [19][67][72]

- **Finish with the two tests that do prove spending authority.** `z_exportkey` fails on an address held only as a viewing key, and the small test spend described below is the definitive practical proof. Use `z_validateaddress` to rule the destination out quickly, not to rule it in.

Note that `z_validateaddress` documents `address_type` values of `p2pkh`, `p2sh`, `sprout`, and `sapling` only, so it is of limited help when the second-stage destination is a unified or Ironwood address. [72]

Before deleting anything, verify that the expected Sapling balance is visible and spendable. The safest practical test is to send a small amount from that Sapling balance to the modern wallet you intend to keep. Wait for it to confirm, then check that it appears there.

Move the funds to the wallet you intend to keep

For the migration path in this guide:

Sprout → Sapling in the legacy wallet → Ironwood in a current wallet

- *Back up the modern wallet first.*
- *Send a small test amount and wait for confirmation.*
- *Then move the remaining Sapling balance.*
- *Check the modern wallet shows the expected confirmed balance before retiring the legacy setup.*

Keep a small offline recovery archive

Keep at least one encrypted offline copy of:

- `wallet.dat`, or the exported Sprout spending key if that was your recovery material;
- the wallet passphrase, stored separately;
- the `z_getmigrationstatus` output;
- the migration transaction IDs and destination address;
- the sidecar version and approximate final block height.

If `unmigrated_amount` is greater than zero, keep the Sprout key material. When NU7 disables V4 (Mainnet target Nov 5th 2026), any remaining Sprout value will be frozen to today's spending paths; any future retrieval facility would still require your old key material. [11][79]

Shut down cleanly, then remove the temporary node data

```
zcash-cli stop
```

Wait until the sidecar process has fully exited before deleting files or unmounting the disk.

After the funds are confirmed in the modern wallet and the archive above is safe, you can delete:

- the sidecar blocks and chainstate;
- the Zebra state database, if this machine was only for recovery;
- downloaded `datadir` snapshots;
- Sprout proving parameters;
- temporary logs, RPC cookies, and recovery-only binaries.

When can I delete `wallet.dat` from the recovery machine?

Only when all of the following are true:

- `unfinalized_migrated_amount` is 0 ZEC;
- the migration destination is the address you expected;
- the migrated Sapling funds have been moved to the modern wallet;
- the modern wallet backup has been verified;
- no other transparent or shielded funds remain in the legacy wallet;
- you have retained any Sprout key material you may need later.

Recommended final state: remove wallet.dat from the live recovery machine, but retain an encrypted offline archival copy. If the drive will leave your control, use whole-device secure erase or cryptographic erase rather than ordinary file deletion.

Source note

Fields and the 10-confirmation finality rule: [z_getmigrationstatus](#).

Destination and sub-0.01 ZEC stopping behavior: [z_setmigration](#).

PART II

Technical migration reference

Architecture, transaction formats, proofs, wallet state, privacy leakage, and failure modes.

6. Pool and transaction architecture



Pool	Core operation	Proof / authorization	Transaction formats	Key migration consequence
Sprout	JoinSplit descriptions	Groth16 JoinSplit proof; JoinSplit signature; Sprout nullifiers	V2-V4 formats; current Sprout spending uses the V4 format	No Sprout component exists in the V5 format or V6 format. A Sprout spend must be built in the V4 format.
Sapling	Separate Spend and Output descriptions	Groth16 proofs and RedDSA-style authorization	V4 format, V5 format, and V6 format	Can coexist with Ironwood in one V6 format transaction.
Orchard	Actions, each with spend/output sides	Aggregated Halo 2 proof and spend authorization	V5 format and V6 format	Permanently closed to new deposits and internal transfers after the 2026 soundness bug; existing value exits only through the turnstile into Ironwood.
Ironwood	Orchard-protocol Actions in a separate pool	Same Action encoding/Halo 2 family, recoverable note plaintext	V6 format	Separate tree, nullifiers, and balance; forward destination.

What a JoinSplit proves

A Sprout JoinSplit is a Zcash-specific zero-knowledge statement, not a Bitcoin legacy feature. The prover demonstrates several things at once: that selected old notes exist under a published Sprout tree root, that the prover may spend them, that nullifiers are correctly derived, that value is conserved, and that new note commitments are well formed. It reveals none of the notes, keys, recipients, or shielded values.

```
public: anchor | nullifiers | commitments | vpub_new | proof
private: notes | spending keys | Merkle paths | note values | recipients
```

The node separately checks that each nullifier has not already appeared. This is why a replacement recovery wallet needs both historical scanning/witness construction and proof generation, while Zebra or Zakura only needs verification and consensus state.

6.1 Turnstiles and privacy



A turnstile is Zcash's value-pool accounting rule. The chain tracks how much value enters and leaves each pool, so a bug in one shielded protocol cannot silently create unlimited spendable ZEC across the rest of the system. Each pool-crossing transaction publicly adjusts the balances of both pools. The world can see the amount transferred, even when it cannot see the sender, recipient, or shielded notes. [2][10]

Revealed does not mean fully traced

For Sprout → Sapling, the crossing amount is public, but the Sapling output itself remains shielded. For Sprout → transparent, the output is a public UTXO and its later spend is explicitly traceable.

Pool balances and the turnstile

In a Sprout withdrawal, `vpub_new` carries the public amount leaving Sprout. In the V6 format, Sapling and Ironwood components each expose a value balance. Consensus verifies that the transaction and block balance and that no pool can be driven below zero. [2][10]

Canopy's exact effect

ZIP 211 requires `vpub_old = 0` for every JoinSplit after Canopy. Net value cannot enter Sprout from non-Sprout sources. Internal Sprout change can still be created, preserving privacy for subsets of the notes being consumed. [2]

How the official migration tool tried to blend in

- It sent migration transactions on a shared block schedule rather than immediately after a user clicked a button.
- It used a shared randomized amount distribution so wallet totals were harder to reconstruct as a unique subset-sum.
- It created exactly one Sapling output and prohibited transparent inputs/outputs and Sapling spends in the migration transaction.
- It allowed less than 0.01 ZEC to remain if that avoided a uniquely identifying final transaction.
- `zcashd` 6.12.0 accelerated the schedule from up to five transactions every 500 blocks to up to five every 100 blocks. It also raised the average amount for balances above 1 ZEC, from roughly 5 ZEC to 500 ZEC. The change shortened the period users had to keep an obsolete node running. That addressed a usability and potential loss-of-funds risk, while keeping randomized, multi-transaction migration. [6][25]

The original ZIP 308 simulation estimated a median of about 1.46 days for 1 ZEC, 2.69 days for 100 ZEC, and 6.95 days for 1,000 ZEC. Those historical values describe the original conservative schedule, not the accelerated 2026 parameters. The migration may still take multiple rounds and should not be interrupted merely because it looks eccentric. [4][6]

The less-than-0.01 ZEC remainder is not a minimum transaction fee. It is a migration stopping rule intended to avoid revealing a uniquely identifying final amount. ZIP 308 specifies a 0.0001 ZEC fee for each migration transaction; current manual transactions may follow different proportional fee policy. [4]

7. Sprout → Sapling: what must exist and run

Required components



You may not need any of this. The Argos rescue wallet needs none of the first three: no front node, no sidecar, and no stock-format zcashd datadir. It can read a wallet.dat or standalone Sprout spending key and perform the recovery scan using remote Zcash services and peers. If you have not decided between the two approaches, **settle that in Section 3** before building anything here. [74][75]

Component	Responsibility	Why it is needed
Zebra 6.3.0 or current Zakura (1.4.0 on Sept 18th)	Current network, consensus, validation, relay	Follows the current chain and accepts valid V4 transactions only until NU7 activates. Re-check the installed node release before a deadline-driven recovery. [8][58][79]
zcashd-compatible sidecar (v1.1.0) or reviewed equivalent	Legacy wallet.dat, Sprout scanner, witnesses, prover, V4 format builder	Modern nodes do not construct Sprout spends; v1.0.0 stops at Ironwood activation.
Stock-format zcashd datadir	Historical blocks, chainstate, and indexes used by the legacy wallet	The sidecar validates and maintains its own state; an existing synced datadir saves time.
wallet.dat / Sprout keys	Spending authority and wallet metadata	Standalone Sprout keys may not derive from a modern seed.
Sprout proving parameters	JoinSplit Groth16 proof generation	Verification alone is insufficient; the wallet must generate the proof.
Sapling destination	Receives the migrated shielded value	The official migration transaction always creates one Sapling output.

What recovery material is sufficient

Any of the following may be enough: a complete old computer or unpruned datadir containing wallet.dat; a standalone wallet.dat backup; a z_exportwallet key dump; or an exported Sprout spending key. The old GUI application itself is not required. A public zc address alone is insufficient. Sprout keys generally cannot be reconstructed from a later Sapling or Orchard mnemonic. [19]

What is the constrained zcashd sidecar?

The constrained **zcashd sidecar** preserves the legacy wallet functions needed to recover Sprout funds after stock zcashd stopped operating. It opens an existing wallet.dat, scans for Sprout notes, maintains their witnesses, generates JoinSplit proofs, and creates V4 format Sprout-to-Sapling migration transactions.

The sidecar is not exposed directly to the Zcash peer-to-peer network. It is hard-locked to a current Zebra or Zakura full node, which supplies validated blocks, checks the transactions created by the sidecar, and relays valid transactions to the network.

```

Old wallet.dat
↓
constrained zcashd sidecar
Sprout scanning, keys and JoinSplit proofs
↓
Zebra or Zakura
current consensus validation and network relay
↓
Sapling

```

This arrangement keeps the obsolete wallet machinery isolated while allowing a modern node to enforce current consensus rules. It does not add Ironwood support to zcashd, and it cannot bypass NU7 once the network rejects V4 transactions. Its purpose is narrow: keep the established Sprout-to-Sapling migration functionality usable until the NU7 activation height. [8][9][11][79]

Zebra itself does **not use anything instead of wallet.dat, because Zebra does not contain a wallet.**

Zebra deliberately stores only public blockchain state, not private keys, addresses, notes, or wallet transaction history. Its on-disk state is a RocksDB database containing blocks, UTXOs, shielded commitment trees, nullifier sets, and other consensus data. Zebra's state cache contains no private wallet data.

If wallet.dat is encrypted

A wallet from the 2016-2018 era may be passphrase-encrypted. Encryption changes the operational shape of the migration. The scheduler is not a single action: it signs transactions in batches over hours or days, and every one of those signatures needs the wallet unlocked.

```
# Unlock BEFORE enabling the scheduler. The timeout is in seconds.
# Prefer -stdin so the passphrase does not enter shell history.
zcash-cli -stdin walletpassphrase

# Then enable the migration
zcash-cli z_setmigration true

# Re-unlock before the timeout expires; a later call REPLACES the old timeout
zcash-cli -stdin walletpassphrase

# Lock again as soon as the migration is finalized
zcash-cli walletlock
```

- **The unlock must outlive the migration, not just the command that starts it.** `z_setmigration` only enables the scheduler. If the wallet re-locks partway through, later batches cannot be signed and the migration stalls with value still in Sprout. Choose a timeout that covers the expected run and re-issue `walletpassphrase` before it expires; note that a second call overrides the previous unlock time rather than extending it. [17][68]
- **A locked wallet does not always say so clearly.** Some `zcashd` builds report a locked wallet with a misleading error rather than a plain "wallet is locked" message. On an encrypted wallet, treat any signing or wallet-state failure as a lock problem first. [68]
- **An unlocked wallet is a hot wallet.** For the whole unlock window the decryption key sits in the recovery machine's memory. Keep that machine off the public network apart from its local front node, do not leave it unattended, and call `walletlock` as soon as `z_getmigrationstatus` shows finality. [68]
- **If the passphrase is genuinely lost, so are the funds.** There is no reset, no recovery path, and no support channel that can restore it. Partial recall is sometimes workable with password-recovery tooling, run against a copy of `wallet.dat`. Treat that as a last resort. Never hand `wallet.dat` to a third-party "recovery service." See Stuck? Need help?

Test the passphrase before you plan around it

Unlock once, early, on a copy - before committing disk, time, or a migration schedule. A passphrase you cannot produce today will not become available later, and it is better to learn that before the synchronization than after it.

Why the constrained sidecar survives stock `zcashd` EOL

The sidecar build disables stock `zcashd`'s automatic end-of-support halt. It is hard-locked to one front-node peer, does not listen to the public network, and removes mining RPCs. Everything important to the legacy wallet-wallet.dat semantics, chainstate format, wallet RPCs, ZMQ, Sprout note handling, and migration scheduling-remains. Consensus safety is delegated operationally to Zebra as the network-facing implementation, while the sidecar still validates the blocks it receives. [8][9]

What those constraints do not do

They do not make an invalid transaction valid. At NU7 activation, V4 transactions are rejected. The sidecar can remain running, but Zebra or Zakura will reject every Sprout migration transaction it creates. Mainnet target: Nov 5th 2026; exact height due Oct 20th. [11][79][80]

Recommended conservative deployment



The sidecar has exactly one P2P route: the local front node.

For a high-value recovery, Zebra 6.3.0 plus a source-built or independently verified current sidecar remains the conservative reference setup. Zakura 1.4.0 is the current alternative front node as of Sept 18th, but Zakura releases have short end-of-support windows: 1.4.0 is scheduled to halt at block 3,504,731 around Oct 3rd. Do not plan a November Sprout recovery around that build; install and verify the current supported release at recovery time. [8][9][58][79]

Migration configuration and control

```
# zcash.conf - placeholders only
migration=1
migrationdestaddress=zs...YOUR_SAPLING_ADDRESS...

# Runtime RPC alternative
zcash-cli z_setmigration true
zcash-cli z_getmigrationstatus
```

The destination must be a Sapling payment address accepted by the legacy wallet. The migration RPC is a scheduler, not an immediate “send everything now” operation. It waits for its batch boundary, constructs proofs, broadcasts transactions, tracks expiry and confirmations, and repeats until the spendable Sprout balance is exhausted or a small remainder is left. [4][17][18]

Preflight lessons from a recent recovery

- Use `z_setmigration` or the `-migration` configuration path as the primary method. A manual `z_sendmany` attempt hit legacy transaction-builder failures in the recent case, while the dedicated migration scheduler completed the transfer. [16]
- Install and verify `sprout-groth16.params` before attempting proof generation; discovering it is missing after a long sync can trigger another failure cycle. [16]
- Understand the operations: `rescan` searches existing blocks for your wallet’s transactions; `reindex` rebuilds indexes and chainstate; `txindex` is not required for ordinary wallet recovery. Do not run the heavier operations without a diagnosed reason. [16]
- Stop cleanly with `zcash-cli stop` and wait for the process to exit. Abrupt termination can leave the legacy databases needing repair or reindexing. [16]
- Expect background scheduling. One real migration produced 21 transactions over six days, then stopped with less than 0.01 ZEC remaining as designed. [16]

What each V4 format migration transaction contains

Element	Required migration behavior
Sprout inputs	One or more existing notes selected by the normal wallet algorithm.
Sprout change	May be returned internally to the first selected Sprout address.
Public pool crossing	Only the final JoinSplit has nonzero vpub_new, exposing the amount leaving Sprout.
Sapling outputs	Exactly one Sapling output.
Transparent components	No transparent inputs or outputs.
Sapling spends	None; this is an evacuation from Sprout, not a mixed Sapling transaction.
Fee	ZIP 308 specifies 0.0001 ZEC per migration transaction.
Expiry	Long enough to survive the scheduled batch; historical ZIP 308 specifies a 450-block expiry delta.

The transaction cannot send directly to Ironwood because the V4 format has no Ironwood component, while the V6 format has no Sprout component. A shielded Sprout-to-Ironwood evacuation therefore necessarily uses at least two transactions. [4][10]

8. Sapling → Ironwood: modern wallet migration



Required components

- A wallet with the Sapling spending authority, note database, and synchronized Sapling commitment-tree witnesses.
- V6 format transaction construction with Sapling spends and Ironwood Actions.
- Sapling Groth16 proving support and Ironwood/Orchard-protocol Halo 2 proving support.
- An Ironwood receiver/account and a node or light-wallet backend that serves the data required to scan and broadcast.
- A migration policy: immediate sweep, progressive note prioritization, delayed batches, or migration when ordinary payments create change.

One transaction, two components

V6 format transaction
 Sapling component: $\text{Spend}(s), \text{valueBalanceSapling} > 0$
 Ironwood component: $\text{Action output}(s), \text{valueBalanceIronwood} < 0$
 transparent/orchard: optional or empty
 fee: balances across the whole transaction

V6 format was designed to carry transparent, Sapling, Orchard, and Ironwood components. Sapling-to-Ironwood can therefore be atomic: the source note is spent and the destination note is created in the same transaction. The turnstile reveals the net amount leaving Sapling and entering Ironwood, but no transparent output is created. [10]

Unlike the Sprout leg, Sapling-to-Ironwood has no migration-specific fixed fee. It is priced under ZIP 317: 5,000 satoshis per logical action, with a two-action grace floor. So 0.0001 ZEC is the conventional minimum, not a promise that every cross-pool transaction costs exactly that. Sapling and Ironwood contributions are both counted, and wallet padding or extra actions can increase the fee. [51]

Ways a wallet can expose the operation

Wallet behavior	User experience	Privacy / operational trade-off
One-click sweep	Move all spendable Sapling balance to Ironwood now.	Simple, but a unique total and timing may be easy to match.
Scheduled migration	Create multiple transactions over time using a shared policy.	Better potential blending if many wallets use the same policy.
Spend-oldest-first	Normal payments prioritize Sapling notes and return change to Ironwood.	Gradual and natural, but may leave dormant notes for a long time.
Hybrid	Move a large share now, then drain remainder through normal activity.	Balances urgency, fees, and fingerprinting.

ZIP 315: the privacy baseline, not a migration utility

ZIP 315 defined auto-migration as moving older shielded funds to a preferred pool without user intervention. It defined opportunistic migration as doing the same inside a user-initiated transaction. Its core privacy rules were consent and restraint. A wallet should obtain explicit consent before revealing a cross-pool amount. It must not automatically combine balances from different pools to satisfy a payment. It should minimize unnecessary pool crossing. [37]

How Sapling → Orchard was implemented in practice

Wallet approach	What the wallet did	Privacy achieved - and what remained public
YWallet	Managed Sapling and Orchard inside one account, but its optimizer tried to fund recipients from the matching pool. When a Unified Address offered both receivers, it could divide the payment between Sapling and Orchard to avoid a net pool crossing . A separate Pool Transfer screen let the user choose the source pool, destination pool, amount, and maximum output-note size.	This often avoided the turnstile entirely and made deliberate crossing an explicit action. But a Pool Transfer still revealed the transaction's total net Sapling-to-Orchard amount. Pre-splitting several destination notes improved later spendability; it did not hide the total crossing in that transaction. [38][39]
Zingo	Used automatic note management. During an ordinary user-initiated shielded payment, it could also spend Sapling notes and send value to the wallet's own Orchard receiver in the same transaction.	This was opportunistic migration: it avoided a separate immediate sweep and gradually drained Sapling during normal use. The net amount crossing from Sapling to Orchard was still public, and there was no ZIP 308-style shared amount-and-time schedule. [40]

Historical YWallet note. Older YWallet builds exposed controls that could force a Sapling-only recovery leg, but the maintainer now states that YWallet no longer supports Zcash since the Ironwood update and directs Zcash users to Zkool. Preserve old YWallet seed/key material, but do not start a new Zcash pool migration in YWallet. [32][38][39]

Applying the lesson to Sapling → Ironwood

ZIP 315 remains the baseline for a Sapling-to-Ironwood transfer, but ZIP 318's detailed migration procedure is specifically Orchard-to-Ironwood. A Sapling-capable wallet can adapt the same privacy ideas. That support must be stated and implemented by the wallet; it does not follow automatically from V6 or Ironwood support. A privacy-preserving Sapling flow should:

- obtain informed consent and show that each pool-crossing amount will be public;
- avoid automatically combining Sapling with Orchard or other pools merely to satisfy a payment;
- prefer several common-denomination crossings spread over time, rather than one distinctive whole-balance sweep or random high-entropy amounts;
- separate synchronization and application-open timing from broadcast timing, and offer network-layer privacy where practical; and
- state clearly whether the wallet is providing a private scheduler, opportunistic migration, or only a basic one-shot transfer. [24][37]

Do not confuse output-note splitting with migration splitting. Creating several Ironwood notes inside one transaction can make later payments easier, but the chain still reveals the single total amount crossing from Sapling in that transaction. To

reduce amount linkage, the pool crossings themselves must occur as separate, commonly shaped transactions under a shared or well-documented timing policy. [24][39]

Wallet support is release-specific. “The protocol permits it” does not mean every wallet has shipped the V6 scanner, fee logic, migration UI, and backend compatibility. For a Sprout holder, this remains a second-stage choice made only after the Sapling migration has safely confirmed. Never send the migration output to a wallet that cannot prove current Ironwood support or whose recovery material cannot be imported elsewhere. [27]

Wallet migration status after Ironwood activation

The entries below primarily track Orchard-to-Ironwood migration, because that is what wallet release notes normally advertise. A Sprout holder still needs the constrained sidecar for Sprout-to-Sapling, then a current V6 wallet for the second stage. Before sending the recovered Sapling balance, verify that the installed wallet can scan or import the relevant Sapling account and create an Ironwood output. Ironwood readiness alone does not prove legacy-Sapling support. [24][26][28][29][30][31][32][33][34][49]

Wallet / platform	Ironwood and migration status (checked Sept 18th 2026)	Guide implication
Vizor desktop - current release line	Ironwood-capable desktop wallet with a ZIP 318 migration implementation. Vizor documents saved migration progress/schedule plus stop, resume, and restart controls. Verify the current signed release on the project release page before installing. [30][54]	Strong desktop candidate for long migrations because it can remain open. Still verify that the recovered Sapling key/account can be imported or spent before moving the full balance.
Vizor mobile - current store build	Ironwood-capable mobile line with ZIP 318 support. Mobile and desktop build numbers and background behavior are separate release tracks. [30][54]	Check the installed App Store / Play Store build, migration screen, and legacy-Sapling recovery path on the actual device.
Cake Wallet 6.4.4	6.4.4 is the current release as of Sept 18th. Ironwood support shipped in 6.4.0 and Zcash maintenance in 6.4.1. Cake continues to document Orchard-to-Ironwood movement as automatic, without a separate migration setting or consent button. [29][52]	Current package, but this guide does not rank “more automatic” as better. ZIP 318’s preferred model requires schedule review and consent before Orchard funds leave.
Zkool 6.30.0 stable	6.30.0 stable released Sept 12th. It supports Ironwood, legacy Sapling keys, and all Zcash pools except Sprout. 6.29.0 fixed an Orchard migration stall for totals around 0.005-0.0062 ZEC; 6.30.0 adds Official Ledger UFKV import and V6 signing. [31][36]	Strong candidate for recovered Sapling and hardware-wallet paths. Verify key import and a small test spend before moving the full balance.
Zodl - Android 3.13.0	3.13.0 is live on Android as of Sept 18th. Guided private or immediate migration remains consent-first; after confirmation, scheduled delivery can continue in the background on a best-effort basis where Android permits it. [50][53][82]	Automation comes after consent. Allow background operation if using the private schedule, and re-open the app if the OS suspends it.
Zodl - iOS 3.13.0	3.13.0 is live on iOS as of Sept 18th. Guided private/immediate migration remains available. iOS scheduled sends principally rely on notifications asking the user to reopen the app. 3.11.0 added a banner for small residual Orchard balances. [33][53][82]	Keep this separate from Android: the migration policy is similar, but execution behavior is not. Do not ignore a residual-Orchard prompt after migration.
Zodl desktop - no official build yet	No official Zodl desktop wallet exists as of Sept 18th. ZODL says official software is distributed from zodl.com; ZODL’s Shawn confirmed that zodldesktopwallet.com was a scam/phishing site. [81]	Do not install a “Zodl desktop” download found through search results, ads, forum DMs, or unsolicited support. Use official Zodl mobile releases or a separately verified desktop wallet.
Zingo PC - documented Ironwood desktop line	The desktop line has a private Ironwood migration path. Release reporting changed rapidly after activation; verify the project release page and installed build rather than relying on an old version number. [55][82]	Desktop can be convenient for a long private migration. Confirm the recovered Sapling account/recovery path before sending the whole balance.
Zingo mobile - current store line / 2.0.23-330 beta	Ironwood support is present in the mobile line. By Sept 18th, beta 2.0.23-330 added device authentication and biometric privacy controls plus Mixnet-aware price-fetch behavior. [56][82]	Check the installed store or beta build and the migration UI on the specific platform. Do not assume desktop and mobile behavior are identical.
NozyWallet CLI 2.4.6; desktop beta.6 separate	CLI 2.4.6 remains the documented production line; desktop is a separate beta track. [57]	Operator-oriented rather than the default first recovery choice. Do not describe the desktop beta as equivalent to the production CLI.
Keystone firmware 3.0.8 with a compatible host	3.0.8 is the current firmware release as of Sept 18th. Ironwood signing still depends on a compatible host wallet and migration flow. [48][84]	Not a Sprout wallet. Verify firmware, host-wallet version, derivation path, and recovery material before moving funds.

Wallet / platform	Ironwood and migration status (checked Sept 18th 2026)	Guide implication
Ledger / other hardware paths	Zkool 6.30.0 adds Official Ledger UFKV import, account creation, and V6 signing with the Official Ledger app. Other host-wallet combinations remain version-specific. [31][35][36]	Not a Sprout route. Verify the exact hardware app, firmware, and host wallet before treating the path as production-ready.
YWallet - legacy only; Zcash unsupported	The maintainer now states that YWallet no longer supports Zcash since the Ironwood update and directs Zcash users to Zkool. [32]	Preserve old YWallet recovery material, but do not begin a new Zcash migration in YWallet. Use a current supported wallet.
Other / edge-case wallets	Verify Sept 18th-or-later release notes for both Ironwood support and the specific legacy Sapling key/account type you need to recover.	Ability to receive Ironwood does not prove ability to spend the recovered Sapling balance.

Consent first; automation second

ZIP 318's preferred model is explicit consent followed by automation. Before any Orchard funds leave, the wallet must show the full proposed migration schedule and obtain the user's confirmation. After that confirmation, scheduled transfers should normally broadcast without asking again for each transaction; background execution is best-effort, not guaranteed. [24]

So "most automatic" is not a quality ranking. Cake Wallet 6.4.4 is the current package as of Sept 18th and Cake documents automatic Orchard-to-Ironwood movement with no separate migration setting or button. That is convenient, but this guide should not present silent automation as inherently better, or as the ZIP 318 ideal, when the standard calls for schedule review and consent before funds leave. [29][52]

Zodl 3.13.0 is a clearer consent-first example on iOS and Android: users choose immediate or private migration, review the transfer plan, and confirm it. On Android, scheduled sends can continue in the background on a best-effort basis; on iOS, the app principally notifies the user to reopen it for the next scheduled send. [33][50][53][82]

- **On mobile, a long private migration may require you to reopen the app. iOS does not permit the same background sending behavior as Android. Wallets therefore use notifications and platform-specific scheduling. There is no official Zodl desktop wallet as of Sept 18th; ZODL has explicitly warned that a "Zodl desktop" site was a scam. For unattended desktop migration, use a separately verified desktop wallet such as Vizor, Zingo, or Zkool rather than an unofficial Zodl download. [54][55][81]**

For a Sprout recovery, Orchard-migration polish is secondary anyway. The decisive second-stage test is whether the installed wallet can import or otherwise control the recovered Sapling balance and spend it into Ironwood. Always verify the installed build and recovery path, not only a project's latest release tag.

If you are still holding Orchard, the timing argument has reversed

Early in the Ironwood migration the cautious advice was to wait for a mature private path. That argument has largely reversed. By Sept 18th 2026, 88.6 percent of the Orchard balance sealed at NU6.3 had migrated, with about 411,000 ZEC still in Orchard and about 3.97 million ZEC in Ironwood. The remaining Orchard set is shrinking, so use a current wallet's migration policy rather than inventing a distinctive manual schedule. [77][83]

- **Every major wallet now has a private or ZIP 318 path.** The reason to wait has expired. Zodl's private flow shipped on Aug 8th 2026. The maintainer of the status thread, who had publicly warned users away from it, reviewed the implementation and moved Zodl to ZIP 318 in the table. [77]
- **Take the wallet's defaults.** The one figure that measured privacy hygiene rather than volume was Cipherscan's. In the week to July 30th 2026 it found 82.5 percent of Ironwood migrations failing ZIP 318 conformance, with over 24 percent not attempting a private path at all. That snapshot was taken early, before the largest wallets shipped their private flows, and the headline percentage was never refreshed. Cipherscan now publishes a per-wallet readiness table instead. Treat 82.5 percent as evidence about hand-rolled crossings in the opening days, not as a verdict on the migration as a whole. The lesson still holds: a hand-picked amount or schedule is exactly what makes a crossing identifiable. [77][78]
- **What compliance looks like now. CipherScan's readiness work distinguishes ZIP 318-conformant migration from other private migration designs. ZODL and Vizor have documented ZIP 318 paths; Zkool intentionally uses a different privacy-first design rather than claiming ZIP 318 conformance. "Private" and "spec-conformant" are not the same claim. Check the installed wallet's current release notes and migration screen before acting. [77][78][82]**
- **A conformant crossing has a precise shape.** Cipherscan defines a ZIP 318 migration as a v6 transaction with no transparent inputs or outputs. Its Orchard value balance is positive and its Ironwood value balance is negative, and the magnitude of the Ironwood balance equals the output denomination. Cohorts are grouped on 144-block anchor boundaries, about three hours. That is the shape your wallet should produce without your involvement. [78]

- **And for the Sprout holders this guide is written for: Sprout still held about 22,481 ZEC on Sept 18th 2026. That remaining value now has a real timetable: NU7 Mainnet is targeted for Nov 5th, when the announced V4 disablement would freeze today's Sprout spending route. [79][83]**
- **Denomination splitting only helps if others are doing it at the same time.** Standard denominations blend a crossing into a cohort. If that cohort is small, the split still leaks through timing, count, and shape. This is the same reason the guide argues against elaborate private choreography that misses a deadline. [77]

Use a ZIP 318 wallet, take its defaults, get it done

That is the closing advice of the community status thread, and it is the right summary for anyone still holding Orchard. The privacy question has moved from "how do I cross carefully" to "how long do I stay visible by not crossing at all."

A note for Guarda Shielded users

Guarda's shielded Zcash wallet is a Sapling-era product, not a Sprout one. Guarda's Zcash light wallet supported transparent addresses only when it was profiled in 2018, with shielded support explicitly planned for after Sapling. The shielded Android wallet was announced for community testing in 2019, after Sapling had activated. Shielded ZEC held there is therefore Sapling value. So this section is the route you need, not the Sprout sidecar path in Part II, and none of its disk, proving-parameter, or datadir cost. [69][70]

- **Do not assume the original app still works.** Guarda's current published material does not clearly document shielded ZEC support, and the shielded wallet no longer appears to be actively promoted. Treat it as possibly unmaintained: plan to recover from your seed or exported keys into a current wallet rather than relying on the original app to sync and spend. [69][70]
- **Verify the balance appears before concluding anything.** If the wallet used a non-standard derivation, importing the seed into a modern wallet can show an empty account while the funds still exist. Zkool documents the broadest import surface - 12, 18, 21, or 24-word seed phrases, legacy zcashd shielded extended keys, and unified viewing keys - and is a reasonable first attempt. If the balance does not appear, do not assume the funds are gone; ask on the forum before experimenting further. See Stuck? Need help? [71]

How this differs from Sprout migration

Dimension	Sprout → Sapling	Sapling → Ironwood
Transaction format	V4 format	V6 format
Number of stages to Ironwood	Two: Sprout→Sapling, then Sapling→Ironwood	One
Legacy key import	Often wallet.dat or standalone Sprout key	Usually ZIP 32 account/seed-derived key
Historical scan	JoinSplit ciphertexts + Sprout tree + witnesses	Normal modern Sapling wallet scan
Proofs	Sprout JoinSplit Groth16	Sapling Groth16 + Ironwood Halo 2
Consensus deadline	Hard cutoff at NU7 V4 activation (Mainnet target Nov 5th; exact height due Oct 20th).	Unaffected by the Sprout V4 format cutoff. No Sapling cutoff date... yet.
Software posture	Archaeological recovery subsystem	Forward-looking wallet feature
Privacy standard	ZIP 308: shared randomized rounds and a small-remainder rule.	ZIP 315: consent and minimize-crossing principles; no standard Sapling scheduler. ZIP 318 covers Orchard → Ironwood.

9. Alternative routes and privacy trade-offs



Sprout → transparent → Ironwood

While the V4 format remains valid, the protocol permits a Sprout withdrawal to a transparent address. The constrained sidecar retains the legacy wallet machinery that may construct it, and a modern V6 format wallet can later spend that UTXO into Ironwood. However, a recent recovery found that manual `z_sendmany` attempts failed while the dedicated Sprout-to-Sapling migration worked. Treat this as an emergency, tested fallback, not the default path. [2][9][11][16]

Why copying ZIP 308's pattern is not enough

You can split withdrawals, coordinate timing, and use common amount distributions to make wallet-level grouping harder. But each transparent UTXO has an exact `txid:vout`, and the later Ironwood shielding transaction explicitly names it as an input. The graph link is deterministic, not merely an amount-matching guess.

Historical research showed that transparent-shielded-transparent round trips were frequently linkable using amount and timing heuristics. A direct transparent bridge is even more explicit because the same public outpoint is spent in the second leg. [15]

Sprout → Sapling → Ironwood

Both pool-crossing amounts are public, so an immediate second transaction with the same rare amount can be correlated. However, the intermediate Sapling note has no public outpoint. A wallet can delay, split, combine with other notes, and follow a common migration policy, turning the association into a heuristic rather than an explicit graph edge.

Urgency outranks elaborate choreography near the NU7 cutoff

ZIP 308's privacy strategy works best when many users share the same schedule and distribution. If few Sprout migrations remain, the anonymity set may be thin. A confirmed evacuation before NU7 disables V4 is more important than a theoretically elegant transaction that misses the deadline. Mainnet target: Nov 5th 2026. [4][79]

10. Where Zebra, Zakura, the sidecar, and modern wallets fit



Software	Role	Sprout migration	Ironwood migration
Zebra 6.3.0	Rust consensus/P2P full node	Fronts and validates V4 transactions built by the zcashd-compatible sidecar (v1.1.0). [8][9]	Validates and relays V6; owns no wallet keys.
Zakura 1.4.0 (Sept 18th)	Alternative Rust full node	Can front its constrained zcashd-compatible path and provides snapshot bootstrapping. v1.4.0 halts at block 3,504,731 (~Oct 3rd), so update again before any later recovery. [41][58]	Validates and relays V6.
zcashd-compatible sidecar (v1.1.0)	Constrained legacy zcashd sidecar	Sprout scanner, witness manager, JoinSplit prover, and V4 builder.	No Ironwood wallet support; transparent and Sapling remain available.
lightwalletd 0.5.3	Light-wallet server	Not a Sprout wallet or wallet.dat recovery tool.	Current Ironwood-capable server line. [59]
Zaino 0.8.0	Rust light-wallet indexer	Not a Sprout wallet or V4 builder.	Current release aligned with Zebra 6.3.0. [60]
zallet 0.1.0-beta.3	Official zcashd wallet replacement	Does not replace the constrained Sprout sidecar.	Current beta wallet tooling; beta.3 adds PCZT-related RPC work and security hardening. Still under development and not a Sprout JoinSplit replacement. [62]
zecd 0.6.2	Rust wallet daemon	Does not open Sprout wallet.dat or construct JoinSplits.	Ironwood-capable wallet-server line; 0.6.2 is the Aug 20th security update. Verify backend/mode for the intended use. [61]
librustzcash	Shared wallet/protocol crates	No drop-in Sprout wallet.dat recovery path.	Use the crate versions pinned by the wallet or application being built; do not treat an old "current rc" snapshot as evergreen.
Modern mobile/desktop wallet	Keys, scanning, note selection, proofs, UI	Not a Sprout tool unless it implements a new legacy module.	Natural home for Sapling/Orchard-to-Ironwood migration.

The front node is not the migration tool. It verifies and relays. The wallet or sidecar discovers notes, owns keys, constructs witnesses, generates proofs, and creates the transaction.

Zebra sidecar versus Zakura sidecar

- Zebra 6.3.0 with the zcashd-compatible sidecar (v1.1.0) is the conservative default. The Zcash Foundation pinned this NU6.3-ready sidecar around Ironwood activation, and the current Zebra release is a recommended security update. Zebra documents the compatibility mode and keeps zcashd behind one peer. [8][9][34][43]
- Zakura 1.4.0 is the current alternative as of Sept 18th, with snapshot bootstrapping and its own zcashd-compatible path. Its release policy matters for a deadline recovery: v1.4.0 is scheduled to halt at block 3,504,731 around Oct 3rd, before the Nov 5th NU7 target. Treat published benchmarks as vendor claims, keep an independently operated Zebra node if you want a second check, and install a supported Zakura build at the time you recover. [34][41][58][79]
- A faster front node buys a faster start, not a faster migration. Once either is caught up, the sidecar's scheduler sets the pace: five transactions per 100 blocks since zcashd 6.12.0, blocks about 75 seconds apart. Zakura's benchmark compares against Zebra's raw P2P sync, not a snapshot bootstrap, and this guide recommends snapshots either way (Section 14). [6][9][41]
- What decides the outcome is the sidecar build, wallet.dat integrity, a synchronized stock-format zcashd datadir, proving parameters, and enough time before the NU7 V4 cutoff. Not which implementation relays the blocks. For production, build or verify the sidecar artifact yourself: Zebra's embedded mode pins the sidecar hash, but independent verification remains prudent. [9][79]

11. ZIP 2003: the NU7 Sprout freeze and future retrieval

How do I know whether Sprout is frozen yet?

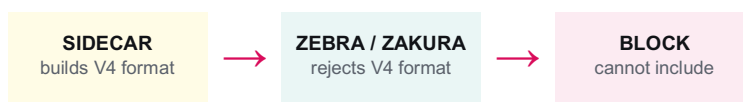
As of Sept 18th 2026, **NU7 Mainnet is targeted for Nov 5th and disabling V4 transactions is in the announced upgrade package**. The exact activation height is not final yet: after the Oct 6th Testnet activation, the Oct 20th Mainnet decision will set it. This guide's ~3,543,000 figure is a planning estimate, not consensus. Before the actual activation height, Sprout can still exit through V4. At or after it, upgraded nodes reject V4 transactions. [11][79][80]

If you are reading this after the actual NU7 activation height and V4 is disabled, **your Sprout coins are frozen in that shielded pool** to today's spending paths. A later consensus upgrade could add a retrieval facility, but none is specified or promised. ZIP 2003 explicitly leaves the Sprout chain value-pool balance issued; disabling V4 is not the same as burning it. [11]

Before activation

V4 transactions remain valid before NU7 activation. Zebra or Zakura can accept a sidecar-created Sprout-to-Sapling migration while V4 remains part of consensus. The dedicated migration route is the best-supported path; manual Sprout-to-transparent transactions are a fallback. Mainnet target: Nov 5th 2026. [10][11][16][79]

After activation



The sidecar process may run, but consensus has the final word.

After NU7 activates with V4 disabled, Sprout funds are frozen to existing spending paths. The sidecar may still open wallet.dat, identify notes, and even attempt to build a V4 transaction, but upgraded Zebra, Zakura, miners, and validators reject it. Existing tools cannot spend a Sprout note into Sapling, transparent funds, or any other destination after that consensus cutoff. [11][79]

What the Future Sprout retrieval facility would require

1. A protocol specification defining the claim, destination pool, nullifier/replay protection, frozen tree state, value conservation, and proof system.
2. Consensus implementations-Zebra, Zakura, and miners' validating software-to implement and activate the new rule at a network-upgrade height.
3. A recovery wallet or standalone tool to import wallet.dat/Sprout keys, scan historical notes, build witnesses, generate the new claim proof, and create an Ironwood or other allowed destination.
4. Testing and auditing that address inflation risk, double recovery, denial-of-service cost, privacy leakage, and legacy key edge cases.

Not every ordinary wallet would need the feature. All economically relevant validating nodes would need the consensus rule, while only recovery-capable wallets would need Sprout key and note support.

Do not confuse two terms

ZIP 2005's formal "Recovery Protocol" is for Quantum recoverable Ironwood notes. The Future Sprout retrieval facility is this guide's label for a separate, hypothetical post-ZIP-2003 Sprout mechanism. [11][12]

Possible designs

Design	Advantage	Main problem
Temporary V4 format window	Reuses the existing sidecar and JoinSplit format.	Reopens the exact legacy attack surface ZIP 2003 sought to remove.
Restricted recovery transaction	Allows exit-only claims without general V4 format.	Still needs Sprout membership, authorization, value, and nullifier verification.
Frozen-state redemption	Claims against the Sprout tree/nullifier state captured at cutoff; could pay directly to Ironwood.	A new circuit/protocol and complex wallet reconstruction are required.
Improved zero-knowledge redemption tooling	A future Sprout redemption circuit might prove ownership and value without generally re-enabling the full Sprout protocol.	No such circuit is implemented or promised; it would still require design, review, audit, consensus deployment, and careful inflation containment.

Better tooling around zero-knowledge proofs might someday make a Sprout redemption circuit easier to implement. But don't expect a design that exposes other pools to unbounded inflation risk by reimplementing Sprout and its transaction format on Zcash mainnet after it has been removed. Most of the possible designs above would make a lot of people very angry and be widely regarded as a bad move. [20][21]

12. Post-quantum recoverability and long-term storage

Sprout retirement and post-quantum planning are related but different. Removing Sprout reduces current complexity and attack surface. Ironwood addresses a different future risk: what happens if practical quantum computers, or another attack that breaks the relevant discrete-logarithm assumptions, force Zcash to disable older pools. [12][20][22]



Pool or destination	Quantum-recoverability status	Migration consequence
Sprout	Not quantum-recoverable	Must leave through a V4 transaction before NU7 activates. Mainnet target: Nov 5th 2026; height due Oct 20th. [11][79][80]
Sapling	Not expected to become quantum-recoverable	Move to Ironwood using the V6 format to gain recoverability.
Existing Orchard notes	Not recoverable under ZIP 2005	Wallets should migrate Orchard value to Ironwood.
Ironwood	Recoverable by design, but not itself fully post-quantum secure	A future Recovery Protocol could recover valid notes into a later post-quantum pool after current protocols are disabled.
Future cold-storage pool	Potentially fully post-quantum; not yet designed	A possible later destination for long-term storage, not a current product or promise.

The V4 format cannot contain Orchard or Ironwood outputs. Only Ironwood notes created in the V6 format, and compatible later pools, are designed to be quantum-recoverable. A Sprout holder therefore needs the two-stage route Sprout → Sapling → Ironwood. [10][12]

Ironwood is a bridge, not the final post-quantum destination. ZIP 2005 defines recoverable note construction and outlines a future Recovery Protocol, but does not make today's payment protocol fully post-quantum. Funds left in Sprout, Sapling, or the old Orchard pool would not be recoverable through that protocol after those pools were disabled. [12]

Tachyon and a future cold-storage pool

Post-quantum security is expected to be considered in Tachyon's design, although Tachyon is unlikely to wait for a post-quantum recursive proof system. It is likely that Zcash will develop a fully post-quantum shielded pool focused on long-term cold storage, but there is no roadmap or commitment yet. Such a pool might need to assume that the party depositing is the party who later withdraws, avoiding some public-key requirements of ordinary payments. [22]

13. Operational checklist and troubleshooting

Before starting

- Make at least **two offline copies** of wallet.dat and record checksums.
- Preserve any passphrase, Sprout spending keys, viewing keys, and notes about imported addresses.
- Do not operate on the only copy of an old disk or wallet file.
- Confirm the final NU7 status and activation height. Sept 18th plan: Testnet Oct 6th, Mainnet target Nov 5th; the final Mainnet decision and exact height are due Oct 20th. [79][80]
- Treat 550 GiB as the combined preflight floor when Zebra state and the zcashd datadir share one mount. For recovery, prefer at least a 1 TiB SSD; use 2 TiB when the compressed snapshot, backups, and extracted datadirs must coexist. [9]
- Obtain the Sprout Groth16 parameter file from an authenticated source and verify its checksum.
- Use Zebra 6.3.0 with the zcashd-compatible sidecar (v1.1.0), or a current supported Zakura release with its own zcashd-compatible mode. Zakura 1.4.0 is current on Sept 18th but halts around Oct 3rd, so it is not the build to carry into a November cutoff window. The sidecar and front-node versions are separate; confirm both before relying on a recovery setup. [8][9][41][58][79]
- Prefer an existing synchronized stock-format zcashd datadir copied after a clean zcash-cli stop, or use an authenticated zcashd-compatible recovery snapshot as described in Section 14. [9]

Before enabling migration

- Verify the front node is fully synchronized and on the expected chain tip.
- Verify the sidecar is synchronized closely behind the front node.
- Verify the sidecar has exactly one P2P peer: the local Zebra or Zakura instance.
- Unlock the wallet only for the necessary duration if it is encrypted.
- Confirm the reported Sprout balance and destination Sapling address.
- Check free space, logs, clock synchronization, and backup integrity.
- Do not substitute manual `z_sendmany` testing for the dedicated migration scheduler. Verify backups, balance, destination, synchronization, and migration status before enabling the scheduler.

During and after migration

- Monitor `z_getmigrationstatus`, generated transaction IDs, expiry, and confirmation depth.
- Expect several scheduled transactions and a possible remainder below 0.01 ZEC. That remainder is a migration privacy threshold, not evidence that the main migration failed.
- Do not assume “broadcast” means “safe”; require mined confirmations well before the NU7 V4 cutoff. Mainnet target: Nov 5th 2026. [79]
- Record the Sapling destination wallet and preserve its recovery material.
- After Sprout evacuation, move Sapling funds to an Ironwood-capable wallet using a deliberate privacy policy.
- Retain archival backups until balances and transaction histories have been independently reconciled.

Common failure modes

Symptom	Likely cause	Action
Sprout balance is zero but expected funds exist	Wrong wallet.dat, wrong chain, incomplete rescan, or viewing/spending key mismatch.	Stop; compare backups and addresses. Do not overwrite the original wallet.
Migration never starts	Initial block download, batch boundary not reached, wallet locked, stale clock, insufficient witness depth.	Check migration status, sync, logs, and wallet unlock state.
JoinSplit proof generation fails	Missing/corrupt Sprout parameters, memory pressure, or sidecar build issue.	Verify parameter checksum; check RAM and logs; use a tested build.
Transaction expires	Slow relay, fee/policy issue, reorg, upgrade boundary, or sidecar lag.	Allow scheduler retry; investigate front-node and sidecar synchronization.
V4 format rejected	NU7 / ZIP 2003 V4 rule active, wrong branch ID, or malformed transaction.	After V4 is disabled, a wallet-only change cannot bypass consensus. This probably means NU7 is live and the Sprout pool is now frozen. See 'ZIP 2003: the NU7 Sprout freeze and future retrieval' .
Sapling funds arrive but cannot move to Ironwood	Destination wallet lacks current V6 format/Ironwood support.	Keep funds in Sapling until a compatible modern wallet is available; do not expose keys casually.
Migration succeeds but less than 0.01 ZEC remains	ZIP 308 stopping rule, not a fee floor	Record the remainder. Do not risk the confirmed migration trying unsupported manual workarounds.
Copied datadir or snapshot will not open	Unclean source shutdown, incompatible state, corruption, wrong paths, or cross-platform database issue	Keep wallet.dat separate; verify manifest/hash; try the Linux reference environment or rebuild indexes from authenticated raw block files.
Migration stalls partway	Some transactions confirmed, then nothing. Wallet re-locked, node lost its peer, scheduler disabled, or the batch interval simply has not elapsed.	Check <code>z_getmigrationstatus</code> first: <code>unmigrated_amount</code> falling means it is still working. Confirm the wallet is unlocked, the front node is at the tip, and <code>z_setmigration</code> is still true. Batches are paced to block height, so a quiet gap of hours is normal.

APPENDIX

Appendices

Recovery snapshots, glossary, command skeleton, migration matrix, and source notes.

14. Copied datadirs and recovery snapshots

The slowest part of many Sprout recoveries is reconstructing the legacy zcashd state. A copied datadir or authenticated stock-format zcashd snapshot can save substantial time, but it supplies only public blockchain state. Keep wallet.dat and private keys separate and local. Do not substitute a Zebra RocksDB state snapshot for the sidecar datadir: the formats are not interchangeable. [9][16]



Direct answer: does Zakura have the missing snapshot?

Yes. Zakura's snapshot page hosts three separate artifacts. Two are Zakura's own Rust chain state, labelled "Pruned" and "Archive." The third, labelled "Zcashd Compat," is exactly the full, unpruned, stock-format zcashd datadir this guide's sidecar needs. It is generated from a "MagicBean"-tagged zcashd build and refreshed roughly daily, usually landing within a day or two of the tip.

There is no separate, first-party Zcash Foundation snapshot to prefer instead. Zebra's own zcashd-compat documentation points operators to this same ValarGroup-hosted artifact. Zebra's community-maintained state snapshots, also from ValarGroup, cover Zebra's own Rust chain state, not the zcashd format the sidecar reads. They answer a different need; they are not a competing source for the same one. [9][41][42]

Starting point	What it contains	Speed and trust trade-off
Copied old zcashd datadir	A cleanly stopped stock-format blocks/index, chainstate, unity data, and possibly the user's wallet	Fastest when healthy and already synchronized. Work on a copy; remove or isolate wallet.dat before sharing anything.
Current zcashd-compat snapshot	Full, unpruned, stock-format zcashd public state (tagged with zcashd's internal "MagicBean" build name) near a declared height. Published at zakura.com/snapshots under "Zcashd Compat," with files served from zcashd.valargroup.dev; refreshed roughly daily and usually only a day or two behind the tip. Both Zebra's and Zakura's zcashd-compat documentation point to this same underlying artifact	Recommended download type for the sidecar. Verify manifest, checksum, height, block hash, and signer.
Zebra or Zakura state snapshot	Front-node RocksDB or implementation-specific validator state. The actively maintained community source is ValarGroup's snapshot service, published at zakura.com/snapshots (the "Pruned" and "Archive" downloads, formerly linked as zebra.valargroup.dev). The Zcash Foundation does not appear to run a separate first-party snapshot host of its own under a "snapshots.zfnd.org" address.	Can accelerate the front node, but cannot be used as the sidecar's zcashd datadir unless explicitly documented otherwise.
Verification snapshot / raw blocks	Authenticated raw block files suitable for local reindexing	Reduces trust in supplied indexes and chainstate, but historical verification can still take days or weeks.

Will the sidecar continue updating it?

Yes. A snapshot ending at stock zcashd's halt height can be opened by the current constrained sidecar. That sidecar connects only to Zebra or Zakura, requests later blocks, validates them, and continues through NU6.3. Use the zcashd-compat sidecar v1.1.0 or a later reviewed build; v1.0.0 stops at the Ironwood activation height. The snapshot is a starting point, not a frozen chain. [9][23][34]

What a signed snapshot proves, and does not prove

A digital signature and checksum authenticate the archive and identify its publisher. They do not prove that the publisher's machine was uncompromised or that the database state was constructed correctly. A strong manifest should include network, height, block hash, sidecar revision, generating node revision, archive size, cryptographic hash, included directories, creation time, and signing-key fingerprint. Independent operators should attest to the same height and block hash.

Filesystem and operating-system portability

Raw blk*.dat block files are the most portable part. Ready-made LevelDB/Berkeley DB indexes and chainstate use the same stock zcashd format, but should not be treated as guaranteed portable across every operating system, CPU architecture, or filesystem. Publish and support a Linux reference environment on a local SSD. On Windows, prefer WSL2 or a Linux VM with the datadir inside its Linux filesystem; on macOS, use a tested native build or Linux VM. Avoid FAT32, network shares, cloud-synchronized folders, and opening one datadir from two systems at once.

Getting a Linux environment on Windows or macOS

Everything above assumes a Linux shell. Here is the fastest supported way to get one before running any command in this guide. Do not run the sidecar, or extract a snapshot, directly on a Windows or macOS filesystem.

You need to avoid FAT32, network shares, cloud-synchronised folders, and other storage whose locking, durability, or filesystem semantics differ substantially from a normal local Linux filesystem. In short your native Windows or macOS filesystem alone **is not up to the job!**

Windows: WSL2

```
# Run from PowerShell as Administrator, then reboot when prompted
wsl --install

# After reboot, open "Ubuntu" from the Start menu and work from that shell
sudo apt update && sudo apt install -y aria2 zstd
```

- Run every sidecar, snapshot, and extraction command in this guide from inside that Ubuntu/WSL2 shell - never from PowerShell or cmd.exe.
- Keep the datadir inside the WSL2 filesystem itself (for example ~/zcash-recovery), not under /mnt/c/.... Crossing the 9p bridge into the Windows filesystem is slow and has been associated with corruption of large RocksDB/LevelDB state.

macOS: a Linux VM

```
# Option 1: Lima - lightweight, command-line Linux VM
brew install lima
limactl start

# Option 2: UTM - full desktop Linux VM with a GUI
brew install --cask utm
```

- Run the sidecar and extract snapshots on the VM's own virtual disk. Do not use a macOS-hosted network share, or a cloud-synced folder such as iCloud Drive or Dropbox. The same portability warning above applies.
- To only stage downloads on the Mac host before moving files into the VM, "brew install aria2 zstd" gets you the same tools used throughout this guide's commands.

Safe snapshot workflow and recommended source

- Create the source archive only after zcash-cli stop has completed and the process has exited, or from a coordinated filesystem snapshot.
- For the sidecar, use only a full, unpruned, stock-format zcashd datadir. The actively updated public source is Zakura's snapshot page (zakura.com/snapshots), under the heading "Zcashd Compat." Files are served from zcashd.valargroup.dev, generated from the same pinned sidecar build described in Section 7. Do not confuse these with the "Pruned" and "Archive"

downloads on that same page. Those are Zakura's own Rust chain state, and cannot stand in for the sidecar's stock-format zcashd datadir. If you would rather anchor to the Zcash Foundation directly, Zebra's own zcashd-compatible documentation links to this same zcashd.valargroup.dev artifact. The Foundation does not run a separate, first-party snapshot service, so there is no independent "ZF-only" unpruned zcashd snapshot to prefer instead. Whichever front node you run, the zcashd-format datadir you need is the same file. [9][41][42]

- Choose the newest authenticated zcashd-compatible snapshot with a declared Mainnet height and block hash. There is no durable single filename to recommend because the archive is periodically replaced. A half-height snapshot is acceptable but needs more catch-up; a newer explicitly compatible snapshot is preferable. Verify the manifest, checksum, signer fingerprint, height, and block hash before extraction. [9]
- Store the compressed archive on a different disk or stream extraction. A 275 GiB-per-datadir minimum does not leave room for both a large archive and its extracted state; 1 TiB combined is the current recommended tier. [9]
- Start Zebra or Zakura and the constrained sidecar without wallet.dat, and allow both to catch up. Confirm that their heights and best-block hashes converge before you introduce the user's wallet.dat or import Sprout keys. [9]
- For stronger assurance, use authenticated raw block files and rebuild local indexes and chainstate with reindexing. This saves network download time, not validation time.

Command and configuration skeleton



Placeholders, not a copy-paste recovery recipe

Paths, RPC authentication, ports, sidecar flags, and destination addresses must match your deployment. Review the current Zebra or Zakura documentation and never put secrets in shell history.

Getting zcash-cli to answer. zcash-cli ships with the sidecar build and talks to the running daemon over local RPC. The daemon must be running and must have finished starting before any command will answer.

```
# zcashd needs a config file to exist. An empty one is fine.
mkdir -p ~/.zcash && touch ~/.zcash/zcash.conf

# Start the sidecar, then check it answers
zcash-cli getblockchaininfo
```

- **Run zcash-cli as the same user, on the same machine.** It finds the daemon and its credentials through the data directory. If you started the sidecar with a custom -datadir, pass the same -datadir to zcash-cli.
- **A connection error means the daemon is not up yet.** Starting takes time before RPC accepts calls. Wait and retry rather than changing configuration.
- **An authentication error means the two are not reading the same config.** Check that you are the same user and pointing at the same data directory, not that your password is wrong.
- **Do not open RPC to the network.** Everything in this guide runs locally. There is no reason to set rpcallowip or bind RPC to a public interface for a recovery, and doing so exposes a wallet-bearing node.


```
# 0. BEFORE syncing: does this wallet hold spendable Sprout? (Section 3)
zcash-cli z_listaddresses
zcash-cli z_exportkey "zc..."

# 1. Start the network-facing node in constrained sidecar mode
zebrad start --zcashd-compat

# 2. Verify synchronization and peer pinning using project tooling
# (exact helper commands depend on the deployment method)

# 3. Query the legacy wallet
zcash-cli getblockchaininfo
zcash-cli getpeerinfo
zcash-cli z_getmigrationstatus

# 4. If wallet.dat is encrypted, unlock for longer than the run (Section 7)
zcash-cli -stdin walletpassphrase

# 5. Enable migration only after destination and backups are verified
zcash-cli z_setmigration true

# 6. Monitor until every intended transaction is confirmed
zcash-cli z_getmigrationstatus

# 7. Verify the destination before deleting anything (Section 5)
zcash-cli z_validateaddress "zs..."

# 8. Stop cleanly before copying any datadir
zcash-cli walletlock
zcash-cli stop
```

For configuration-based operation, zcashd historically accepted `migration=1` and `migrationdestaddress=zs....`. The constrained sidecar retains the legacy RPC/wallet surface, but exact startup wiring is managed by the front node's compatibility mode. Use the migration scheduler rather than a manual Sprout sweep, and verify current project documentation before running commands. [4][9][17][18]

Stuck? Need help?

For migration, node, recovery, or protocol questions, start with the Zcash Community Forum's [Technical Support](#) category. If a new account cannot create a topic, reply to [Post your technical support queries here](#). Read the forum's [Beware of scammers!](#) warning before asking for help. [64][65][66]

- **Search existing threads first.** When you post, include the wallet or node name and version, operating system, the exact error text, and the step you reached - but **strip out secrets and identifying details**.
- **Never post or send** a seed/recovery phrase, private or shielded spending key, wallet.dat, z_exportwallet output, password, PIN, hardware-wallet recovery words, secret QR code, **or anything that can spend your funds**. Never give a stranger remote access to the recovery machine.
- **Do not advertise your exact holdings.** In public, normalize amounts to a standard example such as 1 ZEC or 100 ZEC. Remove addresses, transaction IDs, screenshots, timestamps, and any other detail that could connect your identity to a valuable balance, unless it is strictly necessary to diagnose the problem.
- **Treat unsolicited private messages as hostile until proven otherwise.** Someone who approaches you after a support post, and asks you to "verify" a wallet, visit a recovery site, share a screen, or enter a seed phrase, is not providing legitimate support.
- **Got a wallet-specific problem?** Use the wallet vendor's official support channel reached from the official app, website, or release page. Verify the domain yourself; **do not follow a support link sent to you by an unknown account**.
- **Damaged or partial wallet file?** If a normal restore cannot read the wallet file, ZExCavator from Zingo Labs may help recover usable wallet data or keys. It is a work-in-progress, source-built tool for wallet.dat recovery/forensics. It is not a Sprout migration engine. Once usable spending material is recovered, return to the Argos or legacy-sidecar migration path described in this guide.

It is a dark forest out there. Protect the recovery material first. A legitimate helper can work from sanitized diagnostics; they do not need the secret that controls the ZEC.

Glossary



Anchor - A published root of a shielded pool's note commitment tree. A proof uses it to show a hidden note belongs to the tree.

Bad move - Doing something that makes a lot of people very angry, usually for good reason. Best avoided wherever possible.

Chain value-pool balance - Consensus accounting for net value held in a particular pool.

Cohort - A group of migrations that share an anchor window, and so blend together. CipherScan groups ZIP 318 cohorts on 144-block boundaries, about three hours.

Commitment - A cryptographic representation binding to a hidden note without revealing it.

Datadir / chainstate - The node's on-disk blockchain databases. In this guide, the stock-format zcashd datadir used by the sidecar is distinct from Zebra/Zakura state and is not interchangeable with it.

Denomination - A standard output amount used so that many migrations look alike. ZIP 318 uses fixed denominations to blend a crossing into a cohort.

Fast Recovery Snapshot - An authenticated, ready-made copy of public stock-format zcashd state intended to start the sidecar near a recent block height.

Future Sprout retrieval facility - This guide's term for a hypothetical post-NU7 consensus facility for retrieving Sprout value after V4 spending is disabled. No such facility is currently specified or promised.

Groth16 - The zero-knowledge proving system used by Sprout JoinSplits and by Sapling. Sprout proving is the memory-heavy step in a legacy recovery.

Halo 2 - The proving system used by Orchard and Ironwood. It replaced Groth16 for those pools and needs far less memory than Sprout proving.

Ironwood - The current shielded pool, using the Orchard protocol with recoverable note construction. It is the forward destination for all shielded value, and where every route in this guide ends.

JoinSplit - Sprout's transaction description and circuit: hidden old notes are joined and new notes are split out, with optional public value crossing.

lightwalletd - A server that feeds compact blocks to light wallets. Compact blocks omit the data needed to find Sprout notes, which is why Sprout recovery reads full blocks instead.

Logical action / conventional fee - ZIP 317's public accounting unit for transaction complexity and the fee normally expected by wallets and block producers. The marginal fee is 5,000 satoshis per logical action, with a two-action grace floor.

Mixnet - A network-privacy layer that breaks the link between a broadcast and your IP address. It does not hide the amount, which stays public at a turnstile crossing.

Network upgrade (NU) - A coordinated change to Zcash consensus rules activated at a specified block height. NU6.3 activated Ironwood; NU7 is targeted for Mainnet Nov 5th 2026, with its exact activation height due Oct 20th. [79][80]

NU7 - The next Zcash network upgrade, targeted for Mainnet Nov 5th 2026. Its announced scope includes disabling V4 transactions, which freezes Sprout spending. Testnet targets Oct 6th; the final Mainnet decision and exact height are due Oct 20th. [79][80]

Note - A shielded unit of value controlled by a recipient key; analogous to a private coin record rather than a public UTXO.

Nullifier - A unique public tag derived from a spent note. Nodes reject repeats to prevent double-spending.

Orchard - A shielded pool built from uniform Actions, activated in 2022. Closed to new deposits and internal transfers after the 2026 soundness bug; value now exits to Ironwood.

Proving parameters - Legacy parameter files needed by zcashd to construct Sprout JoinSplit proofs. Missing or corrupt parameters can prevent migration proof generation.

Quantum-recoverable - Designed so that a later Recovery Protocol could prove and recover a valid note after current discrete-log-based protocols are disabled. This is not the same as being fully post-quantum secure today.

Glossary (continued)

Sapling - A lighter, more practical shielded pool introduced in 2018. It is the destination for a Sprout migration and the source for the second leg into Ironwood.

Scheduler - The ZIP 308 migration scheduler inside zcashd. It creates Sprout-to-Sapling transactions in batches paced to block height rather than on demand.

Sidecar - A constrained legacy process run behind a modern front node. Here, zcashd supplies wallet/RPC functionality while Zebra/Zakura supplies network-facing consensus.

Sprout - The original Zcash shielded pool, introduced at launch in 2016. Spends use JoinSplits in the V4 transaction format. It is already exit-only and is targeted to freeze to today's ordinary spending path when NU7 disables V4 on Mainnet, targeted for Nov 5th 2026. [11][79]

Transparent pool - Zcash value held at public t-addresses. Addresses, inputs, outputs, and amounts are all visible on chain, as with Bitcoin.

Turnstile - Pool-balance accounting that reveals net cross-pool amounts and limits inflation leakage between pools.

Unified address (UA) - A single address encoding receivers for more than one pool, so a sender's wallet picks the best one it supports.

UTXO - Unspent transaction output: a publicly visible spendable output in the transparent pool.

V4 format / V5 format / V6 format - Zcash transaction formats. Sprout spending uses the V4 format; Ironwood components are carried by the V6 format.

Verification Snapshot - Authenticated historical block files from which a user rebuilds indexes and chainstate locally, trading speed for reduced trust in supplied database state.

Viewing key - Material that lets a wallet see a shielded address's incoming notes without being able to spend them. zcashd treats such an address as watch-only, and a balance shown from viewing material alone can exceed what you can actually spend.

wallet.dat - zcashd's legacy wallet database. It can contain Sprout and Sapling spending authority and is often the critical recovery artifact for old full-node wallets.

Witness - A Merkle authentication path proving a note commitment is included beneath an anchor.

zatoshi - The smallest ZEC unit. 1 ZEC = 100,000,000 zatoshis.

ZIP - Zcash Improvement Proposal: a protocol, standards, wallet, or process specification.

ZIP 308 - The Sprout-to-Sapling migration procedure and zcashd scheduler. Each migration transaction pays 0.0001 ZEC and the procedure may leave less than 0.01 ZEC unmigrated.

ZIP 315 - Draft wallet best practices that define auto-migration and opportunistic migration, require consent for public information leakage, and tell wallets to minimize pool crossing. It does not specify a complete Sapling migration scheduler.

ZIP 317 - The proportional conventional-fee mechanism used for ordinary Zcash transactions, including the modern stages of the routes in this guide.

ZIP 2003 - The consensus proposal to disallow V4 transactions. It is in the announced NU7 scope. Because Sprout spends require V4, activation freezes Sprout spending. ZIP 2003 does not burn the remaining Sprout pool balance; it leaves that value issued. [11][79]

Scenario matrix



Starting funds	V4 format live?	Recommended path	Required wallet era	Privacy summary
Sprout	Yes - before NU7 activation	Sprout → Sapling → Ironwood	Legacy sidecar, then modern V6 format wallet	Two revealed pool amounts; no public intermediate outpoint.
Sprout	Yes - before NU7 activation	Sprout → transparent → Ironwood	Legacy sidecar, then modern V6 format wallet	Explicit public UTXO link; emergency route.
Sprout	No - at/after NU7 activation	Future Sprout retrieval facility, if ever activated	New recovery wallet + upgraded consensus nodes	Unknown until specified.
Sapling	Not relevant	Sapling → Ironwood in the V6 format	Modern Ironwood wallet	One revealed cross-pool amount.
Orchard	Not relevant	Orchard → Ironwood per wallet migration policy	Modern Ironwood wallet	One revealed cross-pool amount; Action-based pools.
Transparent	Not relevant	Transparent → Ironwood	Modern Ironwood wallet	Public input graph and amount; shielded destination.

Source Notes

Official ZIPs, protocol specifications, project release notes, app-store histories, support documentation, and operator documentation are used for consensus and software-status claims. For rapidly changing wallet, node, indexer, hardware-wallet, and library status, this edition checks primary project and vendor sources through Sept 18th 2026 and asks readers to verify the installed build.

ZIP 315, YWallet's developer documentation, and Zingo's repository are used to distinguish historical Sapling-to-Orchard wallet policy from a standardized migration protocol.

For Sprout deprecation, retrieval constraints, quantum recoverability, Tachyon, and long-term storage, this guide gives particular weight to Daira-Emma Hopwood's ZIPs and factual forum explanations.

For the Orchard vulnerability and its remediation, this guide relies on the Zcash Foundation's and Shielded Labs' own incident writeups.

Community sources are used for the recent recovery case and historical wallet context. ZIP 318 remains a wallet ZIP and wallet releases can change; verify the installed version and the migration screen before acting.

[1] Zcash Protocol Specification. <https://zips.z.cash/protocol/protocol.pdf>

[2] ZIP 211 - Disabling Addition of New Value to the Sprout Chain Value Pool. <https://zips.z.cash/zip-0211>

[3] ZIP 251 - Deployment of the Canopy Network Upgrade. <https://zips.z.cash/zip-0251>

[4] ZIP 308 - Sprout to Sapling Migration. <https://zips.z.cash/zip-0308>

[5] zcashd 2.0.5-2 release notes - official Sprout-to-Sapling migration tool. <https://github.com/zcash/zcash/releases/tag/v2.0.5-2>

[6] zcashd 6.12.0 release notes - accelerated Sprout migration. <https://github.com/zcash/zcash/releases/tag/v6.12.0>

[7] zcashd End of Life. <https://zcash.github.io/zcash/user/end-of-life.html>

[8] Zcash Foundation - Zebra 6.3.0 Release, a recommended security release; operators are encouraged to upgrade. <https://zfnd.org/zebra-6-3-0-release/>

[9] Zebra Book - zcashd-compatible mode, the zcashd-compatible sidecar (v1.1.0), datadir snapshots, and hardware preflight. <https://zebra.zfnd.org/user/zcashd-compatible.html> (this guide could not independently re-fetch this exact page; corroborated

instead via the ZcashFoundation/zcashd repository and the ZF Engineering Update covering the v1.1.0 sidecar pin - see [41])

[10] ZIP 229 - Version 6 Transaction Format. <https://zips.z.cash/zip-0229>

[11] ZIP 2003 - Disallow version 4 transactions. <https://zips.z.cash/zip-2003>

[12] ZIP 2005 - Ironwood Quantum Recoverability. <https://zips.z.cash/zip-2005>

[13] Zcash Swing Wallet repository - GUI wrapper around zcashd tools. <https://github.com/TheTrunk/zcash-swing-wallet>

[14] ZecWallet FAQ - reuse of WinZEC, Zcash4Win, and Zcash4Mac data/wallets. <https://docs.zecwallet.co/faq/>

[15] Quesnelle, "On the linkability of Zcash transactions". <https://arxiv.org/abs/1712.01210>

[16] Nearlybroke's Sprout ZEC recovery thread - recent field case. <https://forum.zcashcommunity.com/t/nearlybroke-sprout-zec-recovery-thread/54996/50>

[17] Zcash RPC documentation - z_setmigration. https://zcash.github.io/rpc/z_setmigration.html

[18] Zcash RPC documentation - z_getmigrationstatus. https://zcash.github.io/rpc/z_getmigrationstatus.html

[19] zcashd wallet backup and key export documentation. <https://zcash.github.io/zcash/user/wallet-backup.html>

[20] Daira-Emma Hopwood on Sprout removal, retrieval risk, and quantum recoverability. <https://forum.zcashcommunity.com/t/do-you-support-deprecating-sprout/49554/122>

[21] Daira-Emma Hopwood on the possibility that improved zero-knowledge tooling could enable a Sprout redemption circuit. <https://forum.zcashcommunity.com/t/do-you-support-deprecating-sprout/49554/74>

[22] Daira-Emma Hopwood on Tachyon and a possible fully post-quantum long-term cold-storage pool. <https://forum.zcashcommunity.com/t/do-you-support-deprecating-sprout/49554/131>

[23] ZIP 258 - Deployment of the NU6.3 Network Upgrade. <https://zips.z.cash/zip-0258>

[24] ZIP 318 - Orchard to Ironwood Migration. <https://zips.z.cash/zip-0318>

[25] zcash issue #7121 - accelerate migration to reduce usability and potential loss-of-funds risk. <https://github.com/zcash/zcash/issues/7121>

[26] Zodi Support - "Moving your funds to Ironwood," describing manual migration; the privacy-preserving guided flow described here as forthcoming has since shipped in 3.9.0/3.9.1 - see [48][50]. <https://support.zodi.com/article/42-moving-your-funds-to-ironwood>

[27] zcash-swift-wallet-sdk pull request #1812 - Orchard-to-Ironwood migration Synchronizer surface, engine, host, and privacy gate. <https://github.com/zcash/zcash-swift-wallet-sdk/pull/1812>

[28] CipherScan block API and explorer - NU6.3 activation block 3,428,143, observed on July 28th 2026 at approximately 14:14 UTC. <https://api.mainnet.cipherscan.app/api/block/3428143>

[29] Cake Wallet releases - v6.4.4 released Sept 4th 2026 and is the current package for this edition; Ironwood support shipped in 6.4.0 and v6.4.1 carried Zcash-specific maintenance. https://github.com/cake-tech/cake_wallet/releases

[30] Vizer Wallet official site / release channel - Ironwood and ZIP 318 migration; verify the current post-activation build. <https://vizer.cash/>

[31] Zkool releases - v6.30.0 stable released Sept 12th 2026; v6.29.0 fixed an Orchard migration stall for small residual totals, and v6.30.0 adds Official Ledger UFKV import/account creation and V6 signing. <https://github.com/hhanh00/zkool2/releases>

[32] YWallet repository - the maintainer now states that YWallet no longer supports Zcash since the Ironwood update and directs Zcash users to Zkool; Ycash remains supported. <https://github.com/hhanh00/zwallet>

[33] Zodi App Store version history - iOS 3.13.0 is the edition-date line; 3.11.0 added handling for small residual Orchard balances after migration. <https://apps.apple.com/us/app/zodi-zcash-wallet/id1672392439>

[34] Zcash Community Forum - "Ironwood is Here! Updated Wallets, Libraries" (updated July 30th 2026), living status thread for nodes, wallets, hardware, and libraries; see [49] for the same thread's later entries through Aug 5th 2026. <https://forum.zcashcommunity.com/t/ironwood-is-here-updated-wallets-libraries-july-30/56557>

[35] LedgerHQ app-zcash repository - supported devices and current PCZT/Ironwood development. <https://github.com/LedgerHQ/app-zcash>

[36] Zkool documentation - supported pools and Shielded Zcash Ledger accounts; all pools except Sprout. <https://hhanh00.github.io/zkool2/> and <https://hhanh00.github.io/zkool2/recipe/ledger.html>

- [37] ZIP 315 - Best Practices for Wallet Implementations: consent, pool-crossing minimization, auto-migration, and opportunistic migration. <https://zips.z.cash/zip-0315>
- [38] YWallet / WarpSync - Transaction Privacy: pool value balances, same-pool funding, and splitting payments across Sapling and Orchard receivers to avoid turnstile crossings. https://hhanh00.github.io/zcash-sync/note_selection/private_tx/
- [39] YWallet 1.2.17 release plus current Pool Transfer, receiver-selection, Custom Send, and Transaction Report documentation - combined Sapling/Orchard account, explicit source/destination pool controls, receiver filtering, output-note pre-splitting, and pre-broadcast pool review. <https://forum.zcashcommunity.com/t/ywallet-beta-1-2-17-is-out/43526> ; <https://ywallet.app/pools> ; <https://ywallet.app/settings/coin> ; <https://ywallet.app/transacting/send> ; <https://ywallet.app/transacting/report>
- [40] Zingolib README - automatic note management and opportunistic movement of Sapling funds to the wallet's own Orchard receiver during a user-initiated shielded transaction. <https://github.com/zingolabs/zingolib>
- [41] Zakura - Zcash Node Snapshots page, listing Pruned, Archive, and Zcashd Compat downloads served from zcashd.valargroup.dev. <https://zakura.com/snapshots/>
- [42] Zakura zcashd-compat documentation - sidecar build provenance, wallet-RPC scope (Orchard and Ironwood permanently unsupported in the wallet; Transparent and Sapling unaffected), and initial-sync guidance pointing to the same snapshot page. <https://github.com/zakura-core/zakura/blob/main/docs/zcashd-compat.md>
- [43] Zcash Foundation - ZF Engineering Update, 27th July to 9th Aug 2026: confirms pinning the NU6.3-ready zcashd-compat sidecar (v1.1.0) around Ironwood activation, and Zebra v6.2.3. <https://forum.zcashcommunity.com/t/zf-engineering-update-27th-july-to-9th-august-2026/56966>
- [44] Zcash Foundation - Zebra 4.5.3 and 5.0.0: Emergency Soft Fork and NU6.2 Activation. Discovery, disclosure, and emergency-upgrade timeline for the Orchard soundness bug. <https://zfn.org/zebra-4-5-3-and-5-0-0-emergency-soft-fork-and-nu6-2-activation/>
- [45] Shielded Labs - Four Questions About the Orchard Vulnerability. Explains why the turnstile, not just the circuit patch, is needed to verify supply soundness. <https://shieldedlabs.net/four-questions-about-the-orchard-vulnerability/>
- [46] Zcash Community Forum - The Orchard Counterfeiting Vulnerability - And Next Steps, by Zooko Wilcox, Jason McGee, and Taylor Hornby. <https://forum.zcashcommunity.com/t/the-orchard-counterfeiting-vulnerability-and-next-steps/56015>
- [47] BlockSec - Zcash Orchard Soundness Bug Analysis: technical detail on the missing equality constraint in the scalar-multiplication circuit. <https://blocksec.com/blog/web3-security-zcash-orchard-soundness-bug-analysis>
- [48] Zodi Support - Migrating to Ironwood (Keystone walkthrough), describing the 3.9.0/3.9.1 "Migrate with Privacy" flow and its Android-automatic / iOS-notified behavior. <https://support.zodi.com/article/65-migrating-to-ironwood-keystone>
- [49] Zcash Community Forum - "Ironwood is Here! Updated Wallets, Libraries," living thread (page 2, entries through Aug 5th 2026): Cake Wallet, Vizor, Zkool, Zodi, and NozyWallet migration status. <https://forum.zcashcommunity.com/t/ironwood-is-here-updated-wallets-libraries-aug-1/56557>
- [50] Zodi - "Zodi 3.9.0: Ironwood Migration & ZODL Slipstream," published Aug 7th 2026. Primary source for the guided migration flow, platform differences, and the Slipstream sync engine. <https://zodi.com/zodi-3-9-0-ironwood-migration-zodi-slipstream/>
- [51] ZIP 317 - Proportional Transfer Fee Mechanism. Defines the conventional fee (5,000-zatoshi marginal fee per logical action, floored at two actions) that prices ordinary Zcash transactions, including the Sapling-to-Ironwood leg of this migration. <https://zips.z.cash/zip-0317>
- [52] Cake Wallet documentation - Ironwood migration: Cake describes the move as automatic, with no migration setting or button. <https://docs.cakewallet.com/cryptos/zcash/ironwood-migration>
- [53] Zodi Support - Moving your funds to Ironwood: consent-first migration plan, Android best-effort background delivery, and iOS reopen/notification behavior. <https://support.zodi.com/article/42-moving-your-funds-to-ironwood>
- [54] Vizor Wallet repository and release channel - desktop/mobile Ironwood migration implementation, saved progress/schedule, stop/resume controls, and signed release guidance. Verify the current release before install. <https://github.com/chainapsis/vizor-wallet/releases> and <https://vizor.cash/>
- [55] Zingo PC releases / Ironwood status - desktop Ironwood private migration path. Release reporting moved quickly after activation, so verify the project release page and installed build. <https://github.com/zingolabs/zingo-pc/releases>

- [56] Zingo mobile release line - Ironwood-capable mobile builds; ZecHub Shielded News Vol.36 notes beta 2.0.23-330 by Sept 18th with device-authentication, biometric-privacy, and Mixnet-aware price-fetch updates. <https://github.com/zingolabs/zingo-mobile/releases> and <https://zechub.substack.com/p/zcash-shielded-news-vol36>
- [57] NozyWallet releases - CLI 2.4.6 is the production line; desktop v1.0.0-beta.6 is a separate beta track. <https://github.com/LEONINE-DAO/Nozy-wallet/releases>
- [58] Zakura changelog - v1.4.0 released Sept 10th 2026; its shortened end-of-support window halts at block 3,504,731, approximately Oct 3rd. <https://github.com/zakura-core/zakura/blob/main/CHANGELOG.md>
- [59] lightwalletd releases - v0.5.3 is the Ironwood-era release cited in this guide; verify the current stable release before deploying. <https://github.com/zcash/lightwalletd/releases>
- [60] Zaino releases - v0.8.0 is the release cited here for Zebra 6.3.0 alignment; verify the current stable release before deploying. <https://github.com/zingolabs/zaino/releases>
- [61] zecd - v0.6.2 is the Aug 20th 2026 security update in the Ironwood-capable wallet-server line. <https://github.com/zecrocks/zecd> and <https://forum.zcashcommunity.com/t/ironwood-is-here-updated-wallets-libraries-aug-20/56557>
- [62] zallet releases - v0.1.0-beta.3 released Aug 26th 2026; beta wallet tooling remains under active development. <https://github.com/zcash/zallet/releases>
- [63] Coinomi Support - "What is the Recovery Tool and how do I export my private keys?" The recovery tool derives coin-specific BIP39/BIP44 addresses and private keys from a Coinomi recovery phrase; for the legacy Coinomi ZEC case discussed here, this is recovery of transparent Zcash keys, not Sprout or later shielded spending keys. <https://support.coinomi.com/support/solutions/articles/29000009717-what-is-the-recovery-tool-and-how-do-i-export-my-private-keys->
- [64] Zcash Community Forum - Technical Support category. General support for Zcash software and recovery questions. <https://forum.zcashcommunity.com/c/support/12>
- [65] Zcash Community Forum - "Post your technical support queries here." New users who cannot create a topic can post a support query as a reply. <https://forum.zcashcommunity.com/t/post-your-technical-support-queries-here/46044>
- [66] Zcash Community Forum - "Beware of scammers!" Forum moderators warn that scammers send private messages to users who report technical support issues, often linking to external sites. <https://forum.zcashcommunity.com/t/beware-of-scammers/46043>
- [67] Zcash RPC documentation - `z_gettotalbalance`, `z_listaddresses`, and `z_exportkey`: combined Sprout/Sapling "private" total, the incoming-viewing-key caution that the reported balance may exceed the spendable balance, and the -allowdeprecated gating of these calls in current zcashd releases. <https://zcash.github.io/rpc/>
- [68] zcashd wallet-lock behaviour - `walletpassphrase` and `walletlock` semantics (timeout in seconds; a later call replaces the previous unlock time), and zcash/zcash issue #3589 on locked wallets reporting a misleading error. <https://github.com/zcash/zcash/issues/3589>
- [69] Electric Coin Company - "Zcash Community Spotlight: Guarda Wallet" (Apr 2018): Guarda's Zcash light wallet supported unshielded addresses only at that time, with shielded support planned after the Sapling upgrade. <https://blog.z.cash/zcash-community-spotlight-guarda-wallet/>
- [70] Zcash Community Forum - "Guarda: Community test for Shielded Wallet" (May 2019), Guarda's announcement of its shielded Android wallet; see also guardaco/guarda-android-wallets issue #15 (2019) for contemporaneous shielded send failures. <https://forum.zcashcommunity.com/t/guarda-community-test-for-shielded-wallet/33671>
- [71] Zkool app listing - documented import formats, including 12/18/21/24-word seed phrases, legacy zcashd shielded extended keys (secret or viewing), unified viewing keys, and BIP-44 derived seed phrases. <https://apps.apple.com/app/zkool/id6736854027>
- [72] Zcash RPC documentation - `z_validateaddress` (fields `isvalid`, `address_type`, `ismine`; `address_type` limited to `p2pkh`, `p2sh`, `sprout`, `sapling`). Wallet membership and spending authority are distinct in zcashd: the wallet may hold an address via an imported viewing key ("watch-only", able to view receipts but not spend, per the 1.0.14 release notes), and the codebase keeps a separate `HaveSpendingKeyForPaymentAddress` check in `wallet.h`. https://zcash.github.io/rpc/z_validateaddress.html
- [73] zcashd -txindex ("Maintain a full transaction index, used by the `getrawtransaction` rpc call", default 0) and the Zcash Insight Block Explorer guide, which notes that `txindex=1` causes zcashd to store all blockchain transactions locally and so requires more storage. https://zcash.readthedocs.io/en/latest/rtd_pages/insight_explorer.html

- [74] Zcash Community Forum - "Zec stuck in old wallet? Use this recovery tool" (Aug 27th 2026), and "Migration from Orchard to Ironwood in Zcash wallets" post 16 (Sept 2nd 2026) confirming that Argos supports migration from wallet.dat files. <https://forum.zcashcommunity.com/t/zec-stuck-in-old-wallet-use-this-recovery-tool/57221>
- [75] Argos Rescue Wallet, Sovright - wallet.dat / standalone-key recovery, Sprout scan behavior, optional donation, and audit information; v1.3.0 released Sept 16th 2026 expands recovery coverage, improves Sapling matching, updates security dependencies, and fixes a sweep that could report success without moving funds to the destination. <https://argos.sovright.com/> and <https://github.com/sovright/argos/releases>
- [76] Network-level privacy during migration - Zcash Community Forum, "Migration from Orchard to Ironwood in Zcash wallets" (wallet developers recommending a network privacy tool for the migration), and Nym, "Free NymVPN for Zcash's Ironwood migration" (July 2026). <https://nym.com/blog/nymvpn-free-for-zcash-ironwood>
- [77] Zcash Community Forum - "Ironwood is Here! Updated Wallets, Libraries (Aug 20)", final update of Aug 20th 2026 and thread discussion: migration progress of about 3.1 million ZEC and 80 percent of the Orchard pool with 95.7 percent supply verification; Zodi's private flow shipping Aug 8th 2026 with Tor protection and its reclassification as ZIP 318; Zkool 6.27.0 adding Nym mixnet transport; NozyWallet's local-zebra and Nym broadcast options; the Zkool maintainer's Tor recommendation; Cipherscan's 82.5 percent ZIP 318 non-conformance measurement of July 30th 2026; and the absence of background sending on iOS. <https://forum.zcashcommunity.com/t/ironwood-is-here-updated-wallets-libraries-aug-20/56557>
- [78] CipherScan - Zcash Ironwood Upgrade and Migration Tracker. The Sept 5th historical snapshot in the previous edition recorded wallet-readiness classifications and migration-shape data; current pool-balance figures in this edition use the Sept 18th ZecStats/CipherScan snapshot in [83]. <https://cipherscan.app/ironwood>
- [79] Zcash Community Forum - "NU7 Timeline," posted Sept 17th 2026 by ebfull / Sean Bowe: Testnet target Oct 6th, final Mainnet decision and activation height Oct 20th, Mainnet target Nov 5th; announced features include 25-second block spacing, disabling V4 transactions, and NSM integration. <https://forum.zcashcommunity.com/t/nu7-timeline/57655>
- [80] Draft NU7 deployment ZIP - NU7 consensus branch ID 0x77190AD8; Testnet and Mainnet activation heights remain TBD until the deployment decision is finalized. <https://zips.z.cash/draft-arya-deploy-nu7>
- [81] Zcash Community Forum - "Zodi Wallet desktop legit?" ZODL's Shawn confirms zoldesktopwallet.com is a scam and says official ZODL software is found at zodi.com. <https://forum.zcashcommunity.com/t/zodi-wallet-desktop-legit/55694>
- [82] ZecHub, Zcash Shielded News Vol.36, Sept 18th 2026 - Zodi 3.13.0 live on iOS and Android; Zingo beta 2.0.23-330 update; Zakura 1.4.0. <https://zechub.substack.com/p/zcash-shielded-news-vol36>
- [83] ZecStats / CipherScan snapshots, Sept 18th 2026 - 88.6 percent of sealed Orchard migrated, about 411,147 ZEC remaining in Orchard, about 3.97 million ZEC in Ironwood, and 22,481 ZEC in Sprout. <https://zecstats.org/ironwood> and <https://zecstats.org/shielded>
- [84] Keystone firmware releases - version 3.0.8 released Sept 15th 2026. Host-wallet Ironwood support remains a separate requirement. <https://github.com/KeystoneHQ/keystone3-firmware/releases>

End operator note and disclaimer



DON'T PANIC - DO VERIFY

The route is understandable: recover the old spending authority, let the constrained zcashd wallet create V4 Sprout-to-Sapling migrations while consensus still permits them, then use a modern V6 wallet to reach Ironwood. The announced NU7 Mainnet target is Nov 5th 2026; aim to be finished and confirmed well before it. [79]

The main dangerous mistakes are working on the only backup, using manual sends instead of the migration scheduler, under-provisioning disk, or waiting until the V4 format has already been disabled.

Your mileage may vary. Keep a verified backup before proceeding. If your only copy of the chainstate is sitting on a network share mounted simultaneously by Windows, macOS, and a Linux VM, this endnote has already failed you. Results are not guaranteed. Past performance is no indication of future performance. No representation or warranty is made, express or implied. Void where prohibited. Professional advice may be required. Batteries not included. Some assembly may be necessary. Objects in mirror may be closer than they appear. Side effects may include data loss, unexpected reindexing, and regret. Use only as directed. Proceed at your own risk.

Change history

Document release history.

Version	Date	Author	Change
v0.1	2026-07-21	TSF	First draft
v1.0 RC	2026-08-20	TSF	Release candidate
v1.0	2026-09-05	TSF	Release – first version
V1.1	2026-09-18	TSF	Release - NU7 deadline and wallet-status refresh